JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Robotics

Obstacle Avoidance

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Robotics

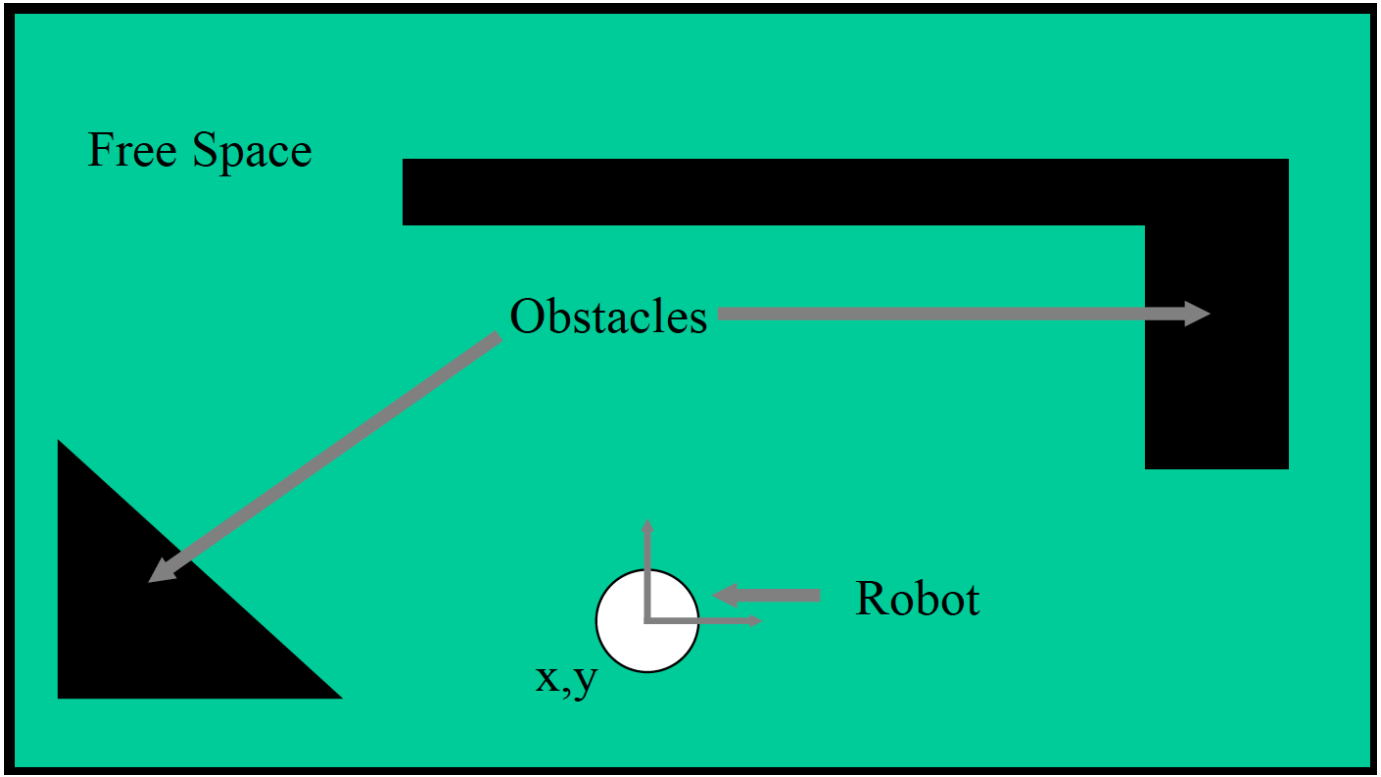
Configuration Spaces

Configuration Space (1)

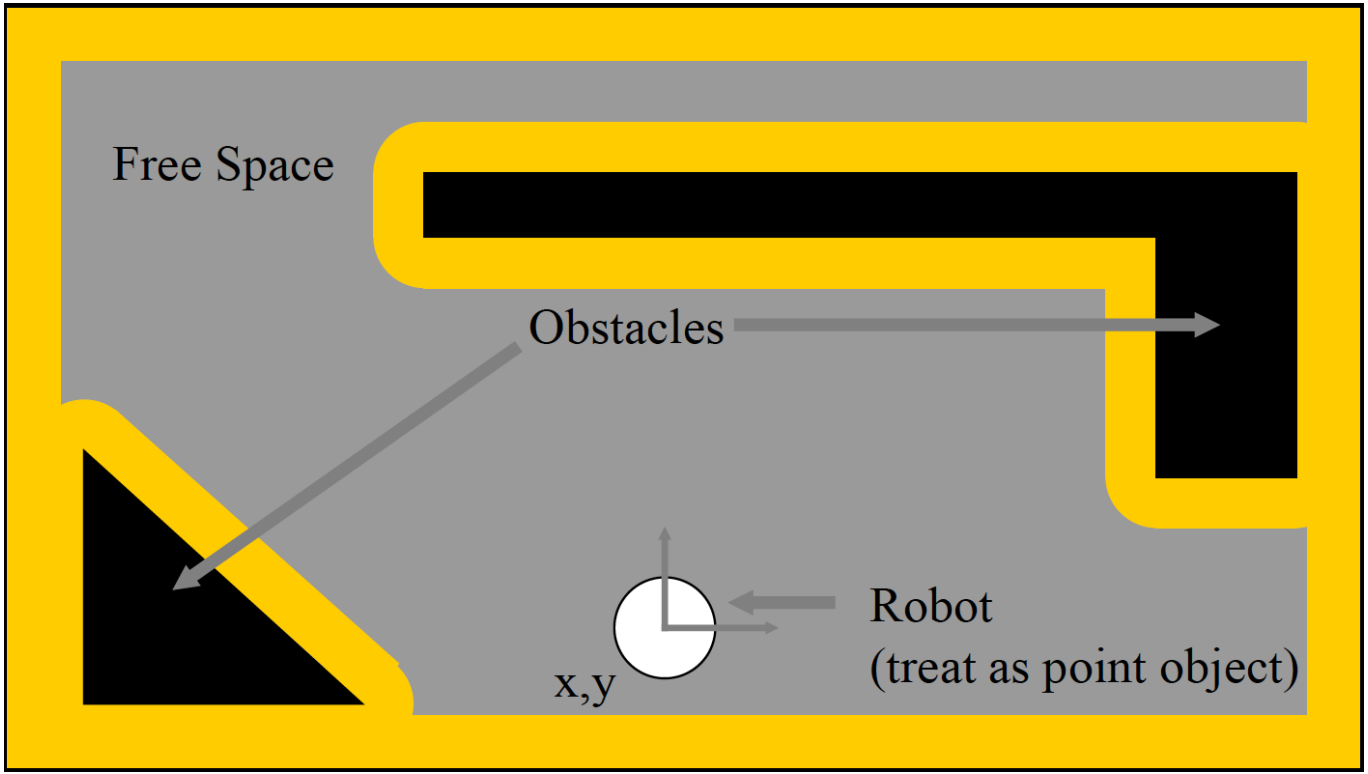
- A key concept for motion planning is a **configuration**:
  - A complete specification of the position of every point in the system

- A simple example: a robot that translates but does not rotate in the plane:
  - What is sufficient representation of its configuration
- The space of all configurations is the configuration space or C-space

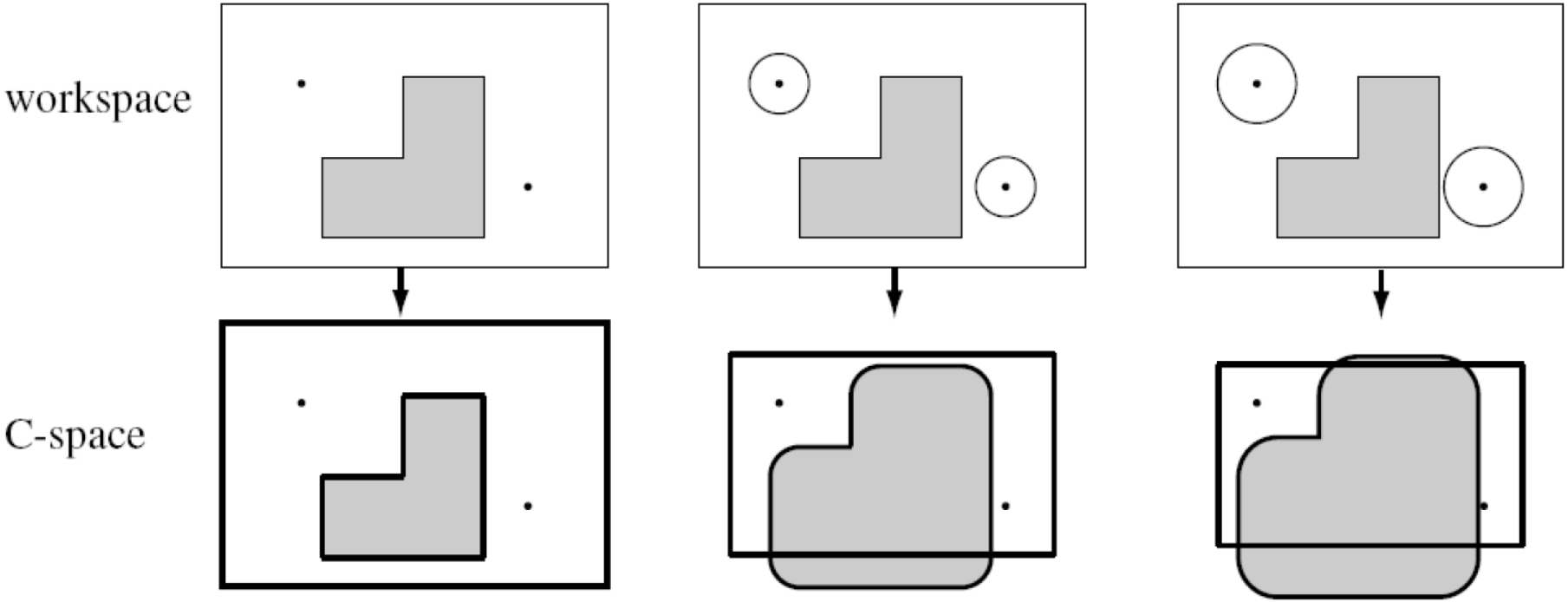
Example of a World (and Robot)



Configuration Space: Accommodate Robot Size



Trace Boundary of Workspace



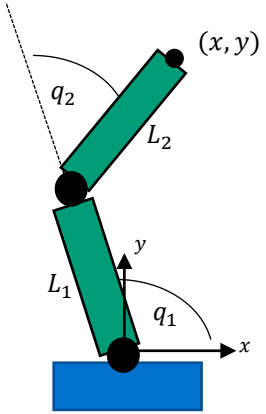
Three Fundamental Issues with C-space

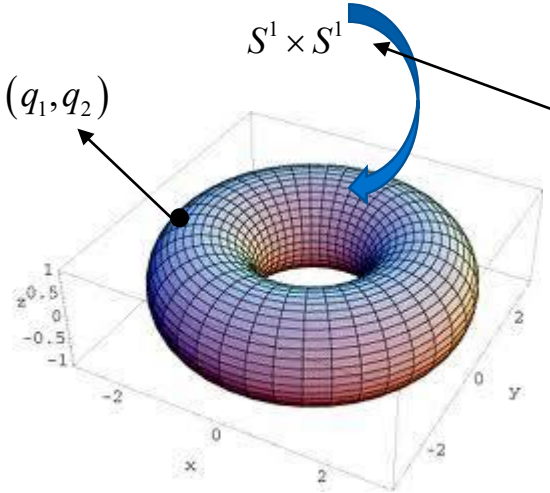
- How do we describe configurations?
  - What is the fundamental space of configurations for a mechanisms?
  - How we relate the configuration space to the workspace? Kinematics
- How are configurations related to controllable inputs to the system?
  - Holonomic vs. Non-holonomic systems
  - Effect of dynamics
- How are obstacles represented in C-space
  - Explicit mapping via Inverse kinematics
  - Implicit in algorithm

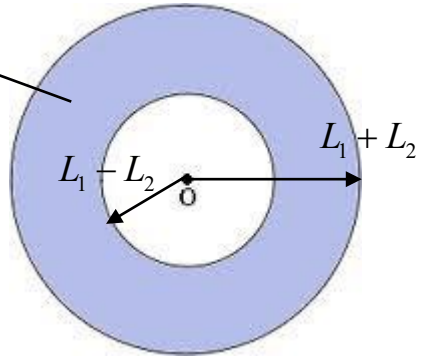
Configuration Space (2)

Cartesian product of two circles (2D torus)

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} L_1 \cos(q_1) \\ L_2 \sin(q_1) \end{bmatrix} + \begin{bmatrix} L_2 \cos(q_1 + q_2) \\ L_2 \sin(q_1 + q_2) \end{bmatrix}$$



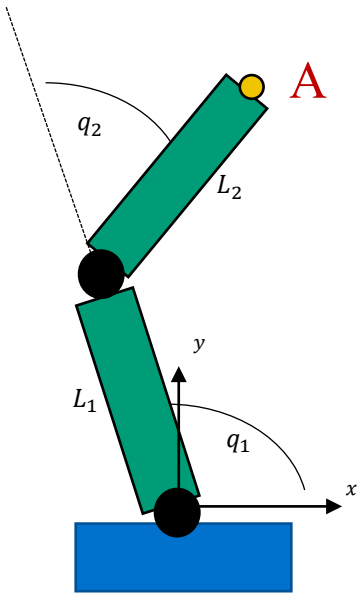


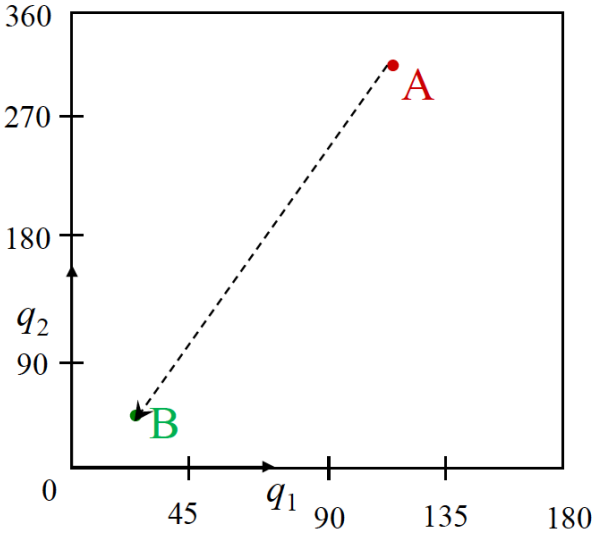


Topology of Configuration Space

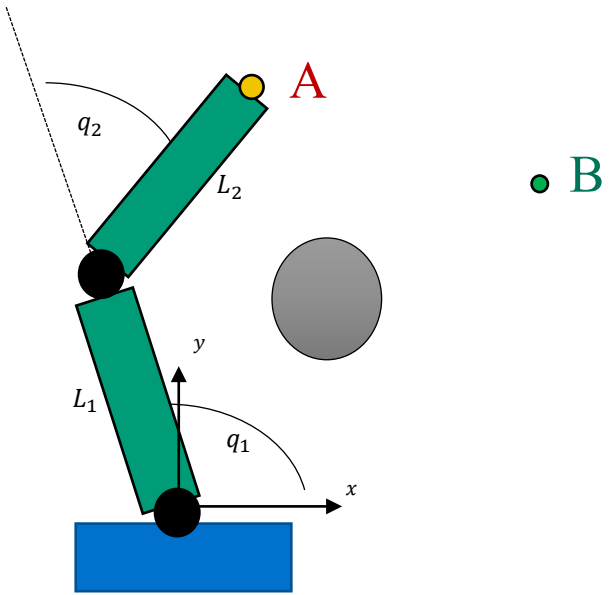
The Workspace

Configuration Space (3)



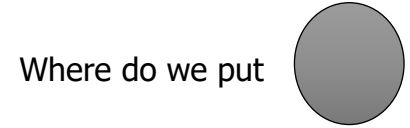


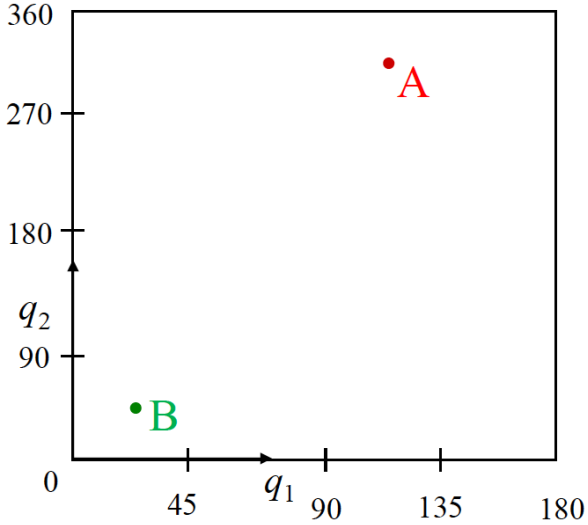
Configuration Space (4)



An obstacle in the robot's workspace

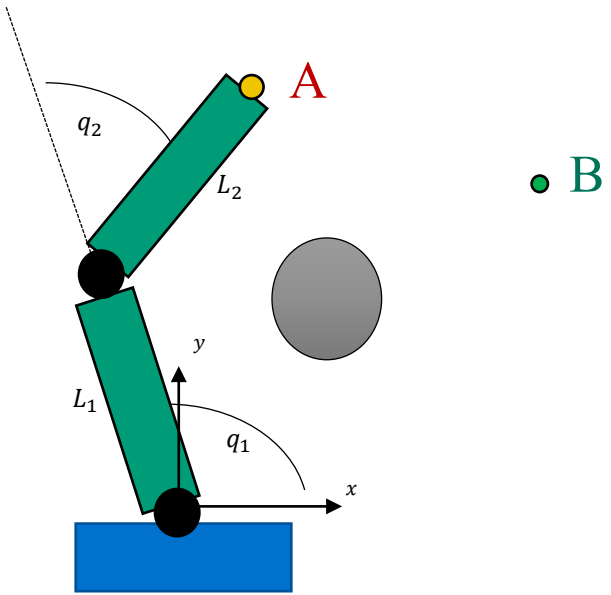
Where do we put



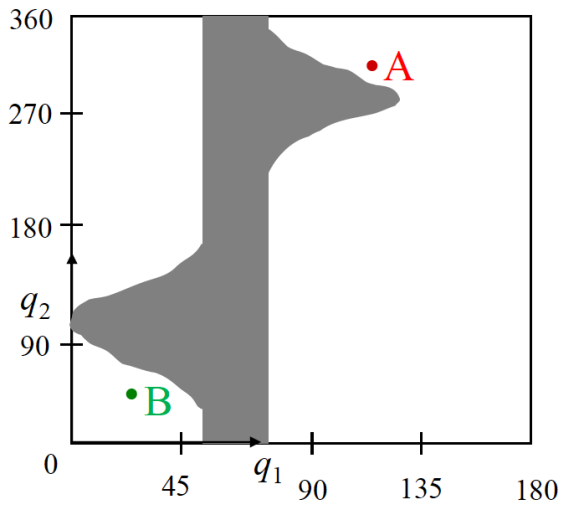


Configuration Space (5)

How do we get from A to B

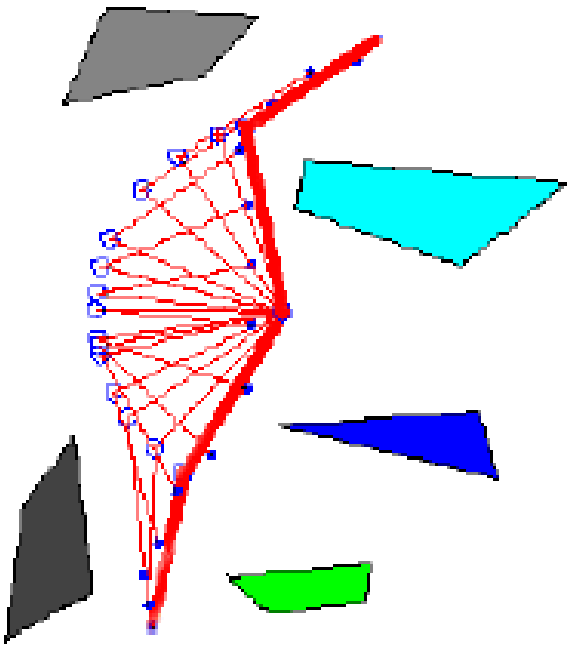


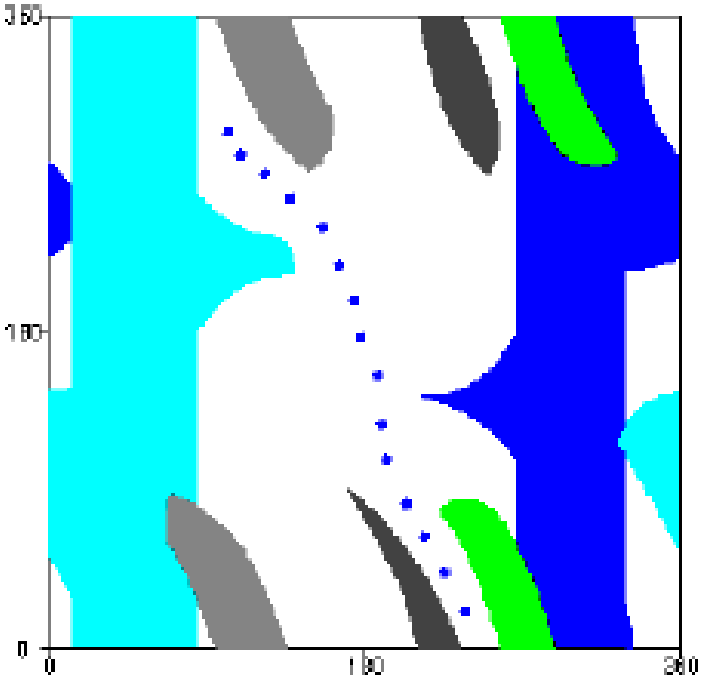
An obstacle in the robot's workspace



The C-Space representation of this obstacle

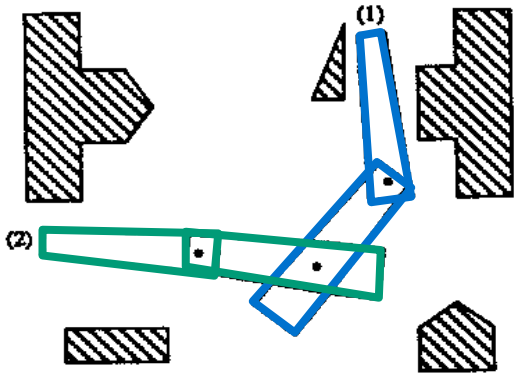
Two Link Path

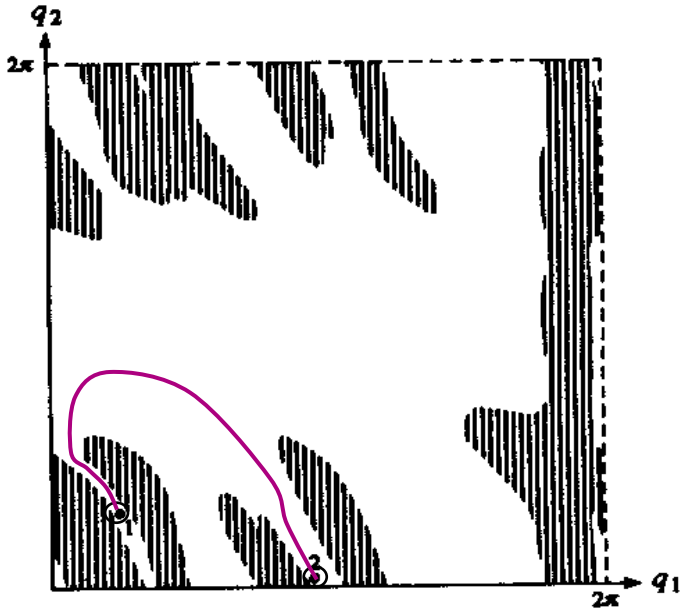




Thanks to Ken Goldberg

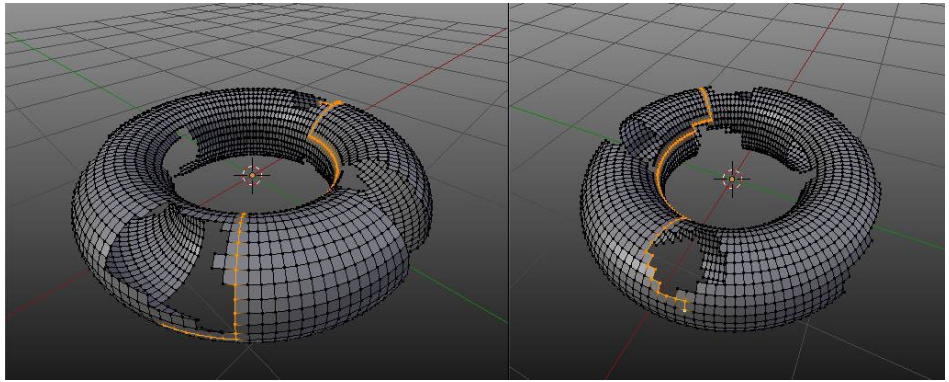
Configuration Space (6)





Some Other Examples of C-Space

- A rotating bar fixed at a point
  - what is its C-space?
  - what is its workspace (the set of points it can reach?)
- A rotating bar that translates along the rotation axis
  - what is its C-space?
  - what is its workspace
- A two-link manipulator
  - what is its C-space?
  - what is its workspace?
  - Suppose there are joint limits, does this change the C-space?
  - The workspace?



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2024, All Rights Reserved.

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

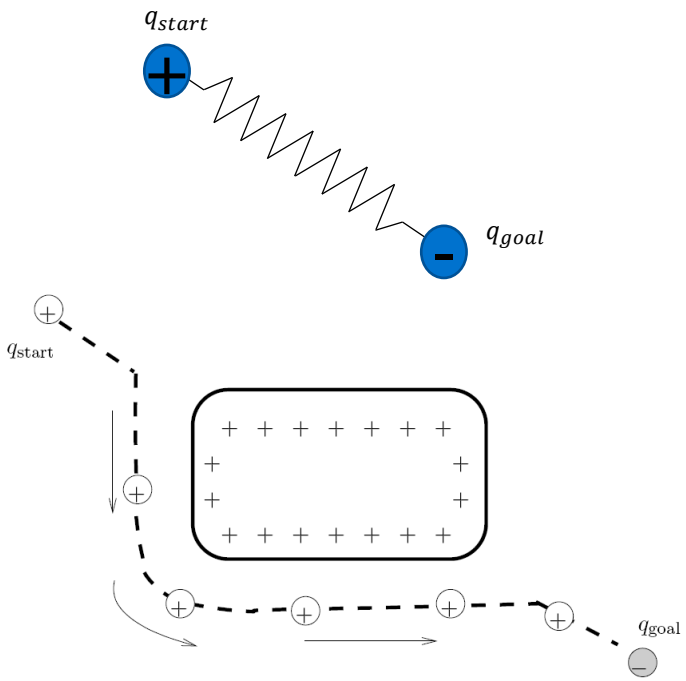
Introduction to Robotics

Potential Fields

Potential Field: The Basic Idea

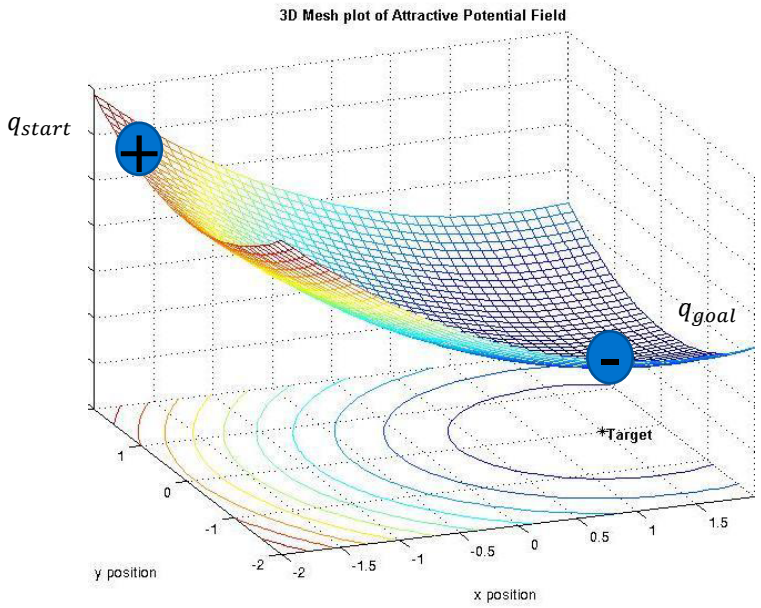
A really simple idea:

- Suppose the configuration goal is a point $\mathbf{q}_{\text{goal}} \in \mathbb{R}^2$
- Suppose the robot is at configuration $\mathbf{q}_{\text{start}} \in \mathbb{R}^2$
- Think of a "spring" drawing the robot toward the goal and away from obstacles:
- Can also think of using same/opposite electric charges to repeal/attract



Another Idea

- Think of the goal as the bottom of a bowl
- The robot is a ball on the rim of the bowl
- What will happen?



mechatronics.eng.buffalo.edu

The General Idea

Both the bowl and the spring analogies are ways of storing **potential energy**

- The robot aims to move to a lower energy configuration

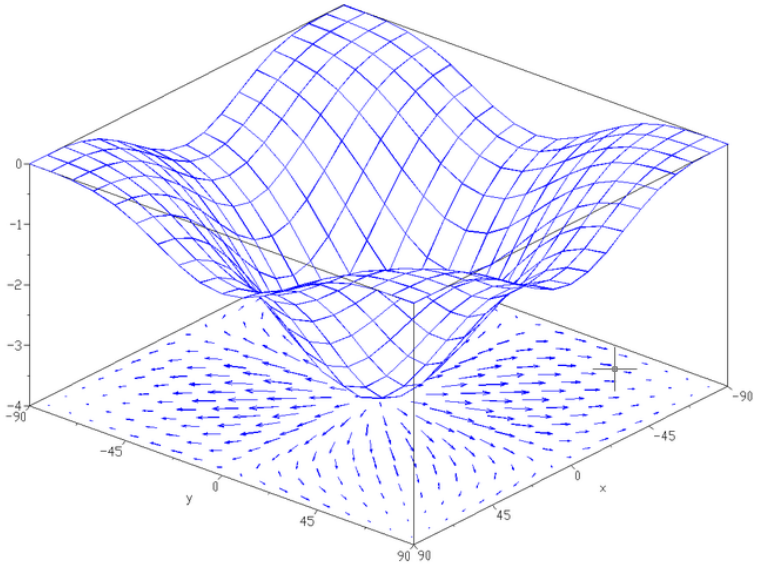
A potential function is a function $U : C \to \mathbb{R}$

Energy is minimized by following the negative gradient of the potential energy function

$$\nabla U(q) = \begin{bmatrix} \dfrac{\partial U}{\partial q_1} \\ \vdots \\ \dfrac{\partial U}{\partial q_M} \end{bmatrix}$$

We can now think of a vector field over the configuration space

At any given point in time the robot looks at the vector corresponding to its current configuration and moves in that direction



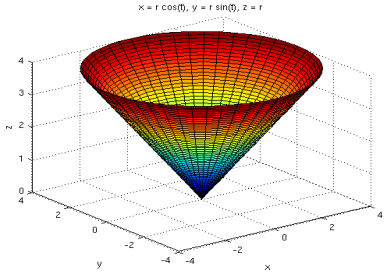
Attractive / Repulsive Potential Field

$$U(q) = U_{\text{att}}(q) + U_{\text{rep}}(q)$$

U_{ATT} is the attractive potential – move to the goal

U_{rep} is the repulsive potential – avoid obstacles

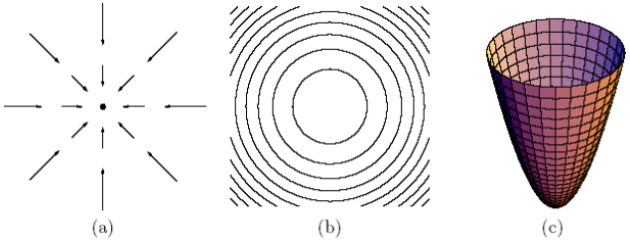
Attractive Potential



$$U_{\text{att}}(q) = \zeta d\left(q, q_{\text{goal}}\right)$$

$$F_{\text{att}}(q) = \frac{\zeta}{d\left(q, q_{\text{goal}}\right)}\left(q - q_{\text{goal}}\right)$$

Quadratic Potential



$$U_{\text{att}}(q) = \frac{1}{2}\zeta d^2\left(q, q_{\text{goal}}\right)$$

$$F_{\text{att}}(q) = \nabla U_{\text{att}}(q) = \nabla\left(\frac{1}{2}\zeta d^2\left(q, q_{\text{goal}}\right)\right)$$

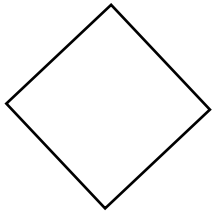
$$= \frac{1}{2}\zeta \nabla d^2\left(q, q_{\text{goal}}\right)$$

$$= \zeta\left(q - q_{\text{goal}}\right)$$

Distance

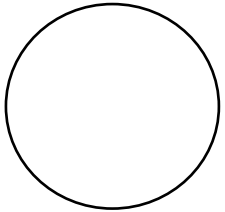
$$d : R^2 \times R^2 \rightarrow R$$

L1 Metric:



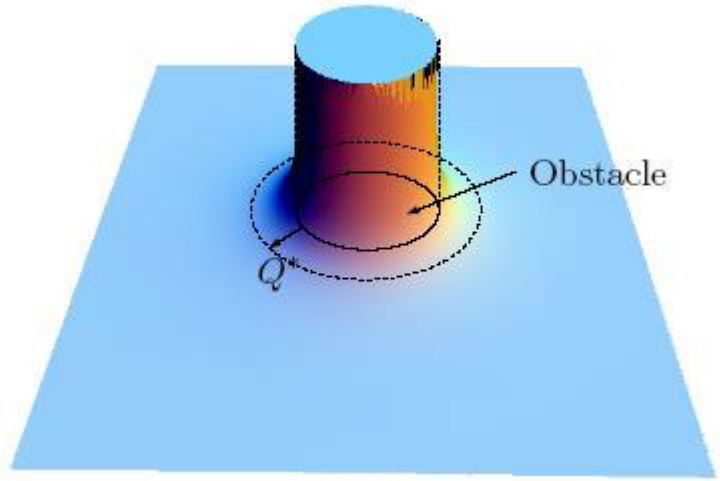
$$d(a,b) = |a_x - b_x| + |a_y - b_y|$$

L2 Metric:



$$d(a,b) = \sqrt{(a_x - b_x)^2 + (a_y - b_y)^2}$$

Repulsive Potential

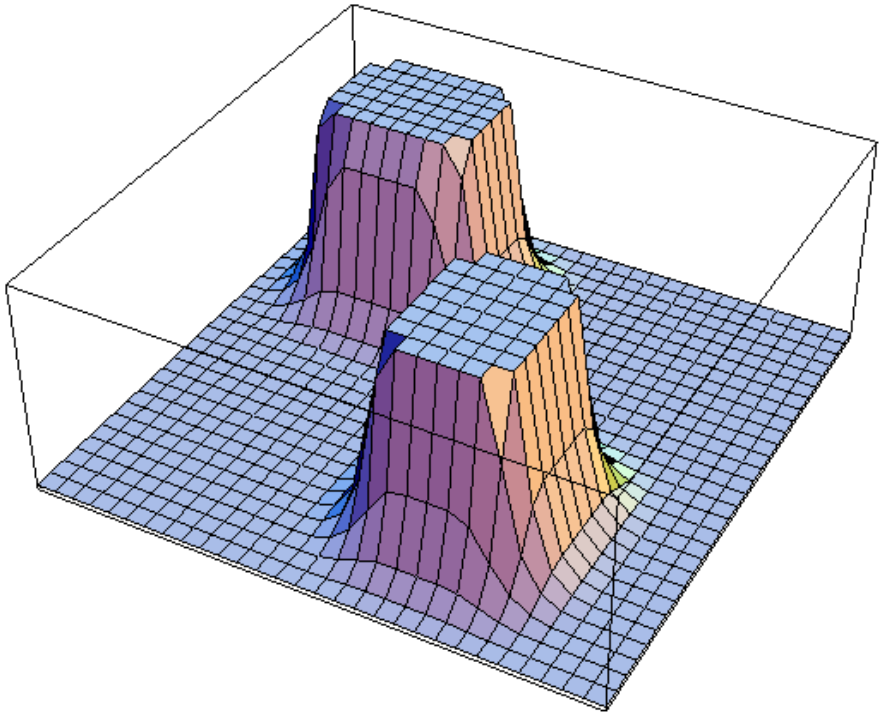


$$U_{\text{rep}}(q) = \begin{cases} 1/2\eta\left(\dfrac{1}{D(q)} - \dfrac{1}{Q^*}\right)^2, D(q) \le Q^* \\ 0, D(q) > Q^* \end{cases}$$

Whose gradient is

$$\nabla Q_{\text{rep}}(q) = \begin{cases} \eta\left(\dfrac{1}{Q^*} - \dfrac{1}{D(q)}\right)\dfrac{1}{D^2(q)}\nabla D(q), D(q) \le Q^* \\ 0, D(q) > Q^* \end{cases}$$

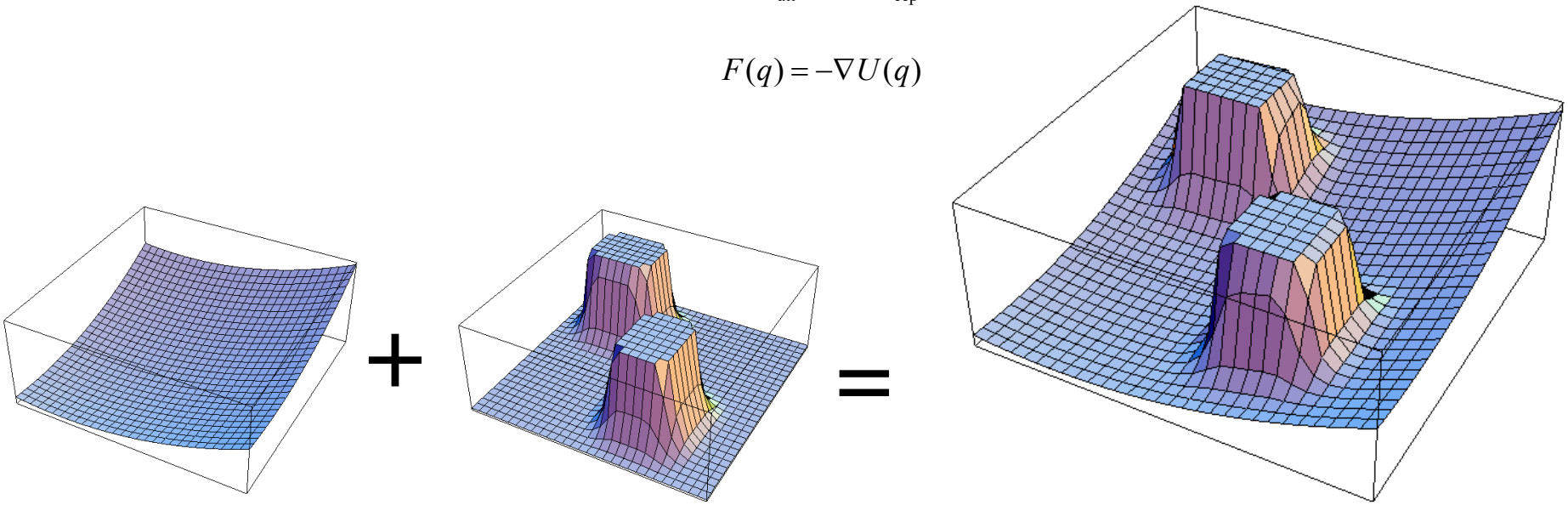
Repulsive Potential (cont.)



Total Potential Function

$$U(q) = U_{\text{att}}(q) + U_{\text{rep}}(q)$$

$$F(q) = -\nabla U(q)$$



Gradient Descent (1)

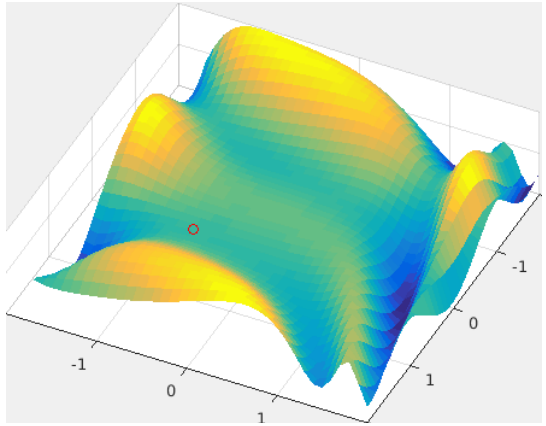
- A simple way to get to the bottom of a potential function $U(q)$

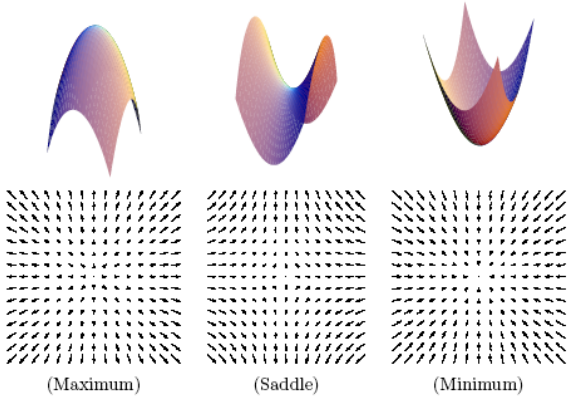
  is to follow the gradient $-\nabla U(\mathbf{q})$
- At each configuration $\boldsymbol{q}$, evaluate

  $-\nabla U(\mathbf{q})$ and take a "small step" $\nabla \mathbf{q}$

  in that direction to reach a new configuration

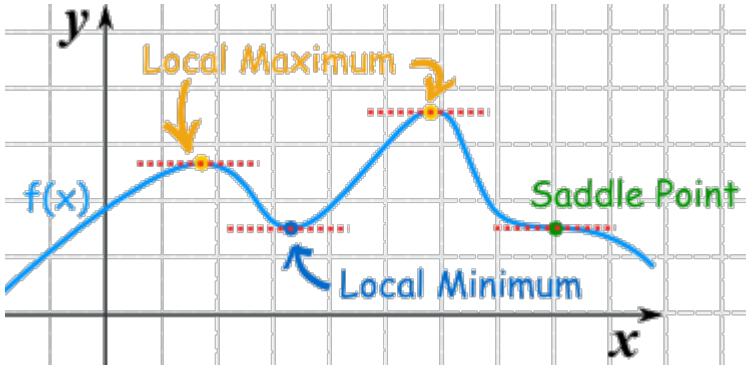
  $\mathbf{q} + \Delta \mathbf{q}$
- At a critical configuration $\mathbf{q}^*, \nabla U \mathbf{q}^* = 0$



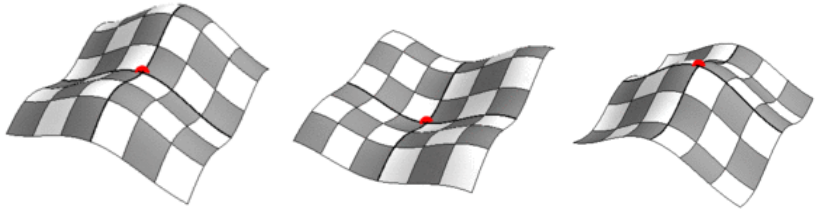


Gradient Descent (2)

- For a 1-d function, how do we know we are at a minimum (or maximum)?
  - 1st derivative tells us if we have a critical point (maximum, minimum, saddle point)
  - 2nd derivative tells us the kind of critical point
    - If $f''x^* < 0$ then x^* is a maximum
    - If $f''x^* > 0$ then x^* is a maximum
- For a M-d function the Hessian $Hf(\mathbf{x})$ is the $M \times M$ matrix of second derivatives
- If the Hessian is nonsingular $\det(H) \neq 0$ the critical point is a unique point
  - if $Hf(\mathbf{x}^*)$ is positive definite $(x^* THf(x^*)x^* > 0)$ $\mathbf{x}^*$ is a minimum (positive eigenvalues)
  - if $Hf(\mathbf{x}^*)$ is negative definite $(x^* THf(x^*)x^* > 0)$ $\mathbf{x}^*$ is a maximum (negative eigenvalues)
  - if $Hf(\mathbf{x}^*)$ is indefinite $\mathbf{x}^*$ is a saddle point



www.mathsisfun.com



Wikipedia.org

The Hessian

- The Hessian is the $M \times M$ matrix of second derivatives

$$H_U(q) = \begin{bmatrix} \frac{\partial^2 U}{\partial q_1^2} & \cdots & \frac{\partial^2 U}{\partial q_1 q_M} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 U}{\partial q_M q_1} & \cdots & \frac{\partial^2 U}{\partial q_M^2} \end{bmatrix}$$

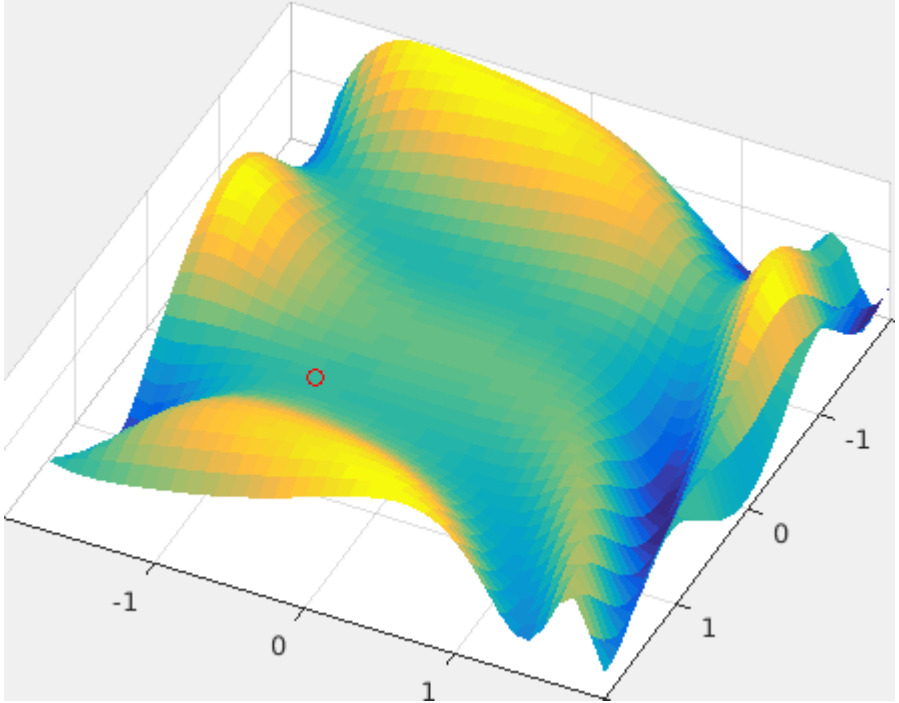
- The Hessian is the Jacobian of the gradient

$$H_U(q) = J_{\nabla U}(q)$$

Gradient Descent (3)

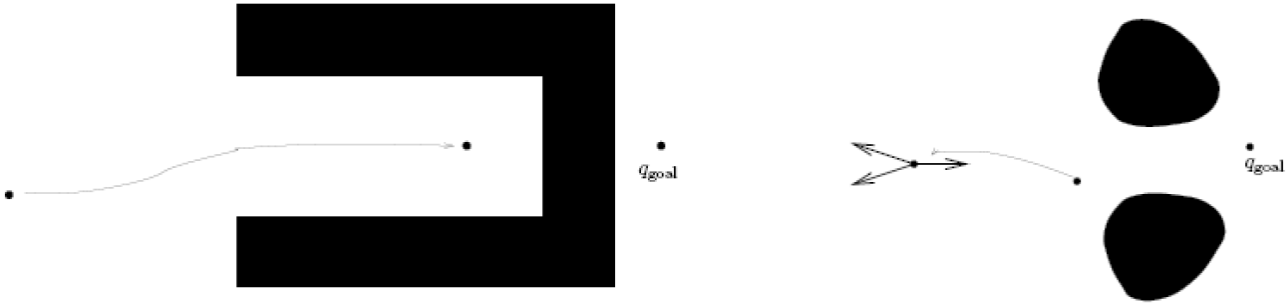
Algorithm

1. You need:
   - A start configuration: q_{start}
   - Gradient of a potential function: $\nabla U q$
2. $q_0 = q_{\text{start}}$
3. $i = 0$
4. while $\nabla U(q_i) \neq 0$ do
5. $q_{i+1} = q_i - \alpha_i \nabla U q_i$
6. $i = i + 1$



Potential Functions Question

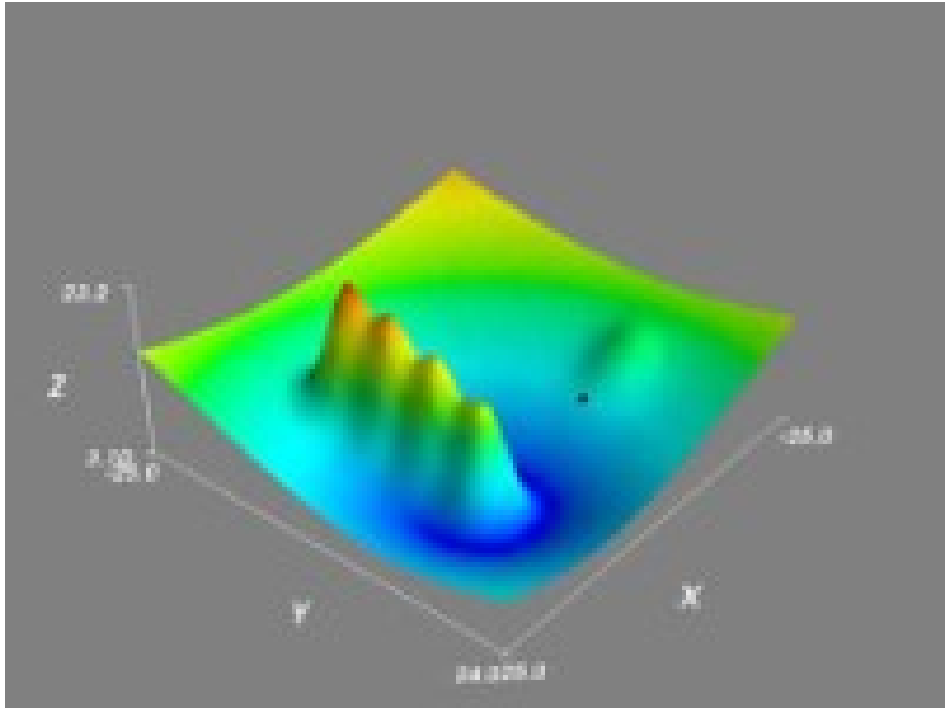
- How do we know that we have only a single (global) minimum?



Principles of Robot Motion

- We have two choices:
  - not guaranteed to be a global minimum: do something other than gradient descent (what?)
  - make sure only one global minimum

Potential Trail Demo



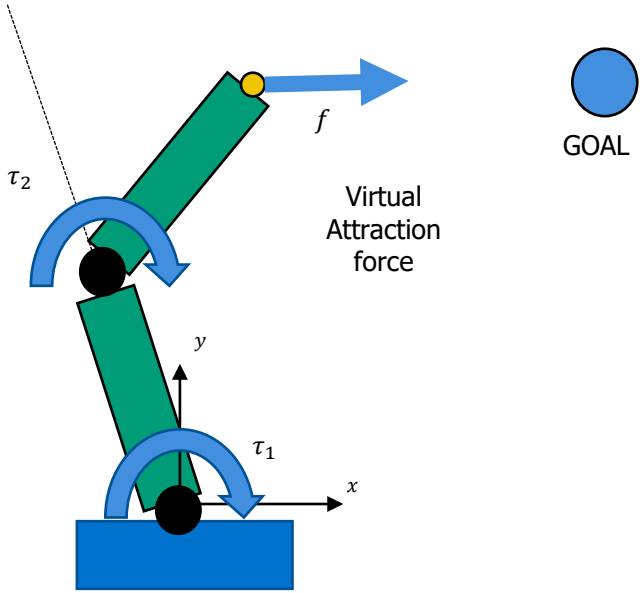
Potential field navigation with local minima avoidance using potential trail. No solution case. YouTube. Raúl Acuña Godoy, 2010

Potential Fields on Non-Euclidean Spaces (1)

- Thus far, we've dealt with points in Rn, which is suitable for mobile robots. What about manipulators?
- Recall we can think of the gradient vectors as virtual forces that attract/repeal a point.
- The basic idea is to define forces in the workspace (which is $\Re^2$ or $\Re^3$

Force $\boldsymbol{f}$ acting at a point $\boldsymbol{p}$ in the robot's workspace
Torques $\boldsymbol{\tau}$ acting in the robot's configuration to move the robot
$\mathbf{v} = J\mathbf{q}$ where $J = \partial f / \partial q$
$\boldsymbol{\tau}^T q$ power in the configuration space
$f^T v$ power in the work space
Power is conserved!

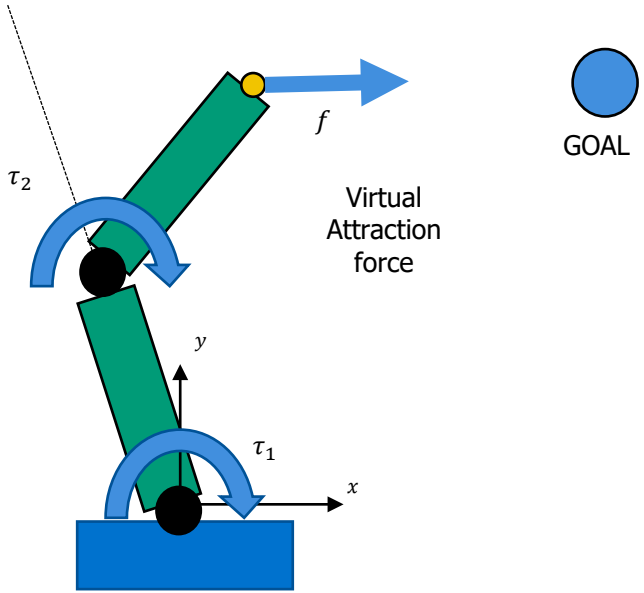
$$f^T J\dot{q} = \tau^T \dot{q}$$
$$f^T J = \tau^T$$
$$J^T f = \tau$$



Potential Fields on Non-Euclidean Spaces (2)

- Recall we can think of the gradient vectors as virtual forces.
- The idea is to define virtual forces in the workspace that will be applied at ***specific points*** on the robot arm
- We have a virtual force ***f*** applied in the workspace (i.e. $\mathbb{R}^2$ or $\mathbb{R}^3$ and its in the configuration space (i.e. $\mathbb{R}^M$
- Using $J^T f = \tau$

  we can define virtual forces in the workspace and then apply the resulting real torques in the configuration space
- Example: our two-link manipulator



Potential Fields on Non-Euclidean Spaces (3)

- Example: our two-link manipulator

$$\begin{bmatrix} x(q_1,q_2) \\ y(q_1,q_2) \end{bmatrix} = \begin{bmatrix} L_1\cos(q_1)+L_2\cos(q_1+q_2) \\ L_1\sin(q_1)+L_2\sin(q_1+q_2) \end{bmatrix}$$

Compute and evaluate $J(q_1,q_2)$

Suppose $p_{\text{goal}} = \begin{bmatrix} x_{\text{goal}} & y_{\text{goal}} \end{bmatrix}^T$ then

$$f = p_{\text{goal}} - \begin{bmatrix} x & y \end{bmatrix}^T$$

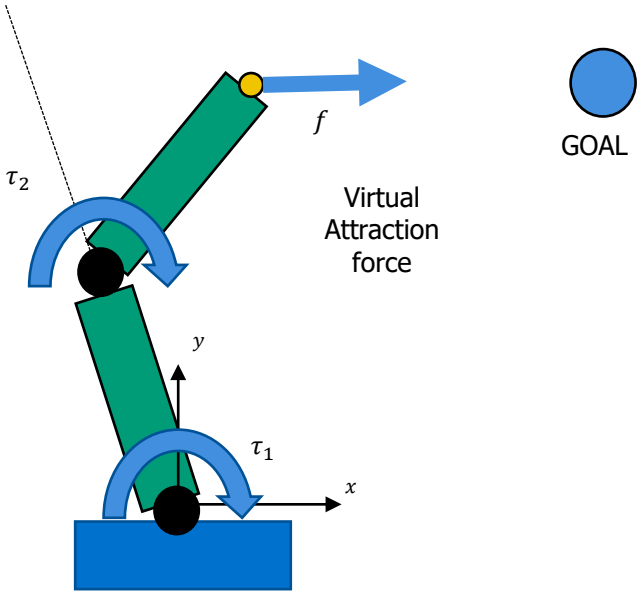
And

$$\tau = J^T(q_1,q_2)f$$

Then apply the joint torques (τ_1,τ_2) to the actuators

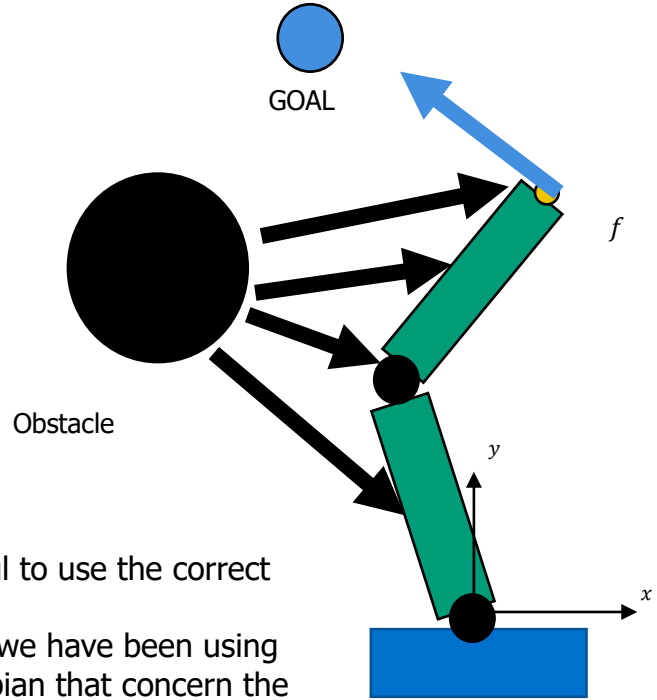
These torques will result in a force f at the end effector that will move the robot toward p_{goal}

If joint *i* is prismatic then τ_i is a force



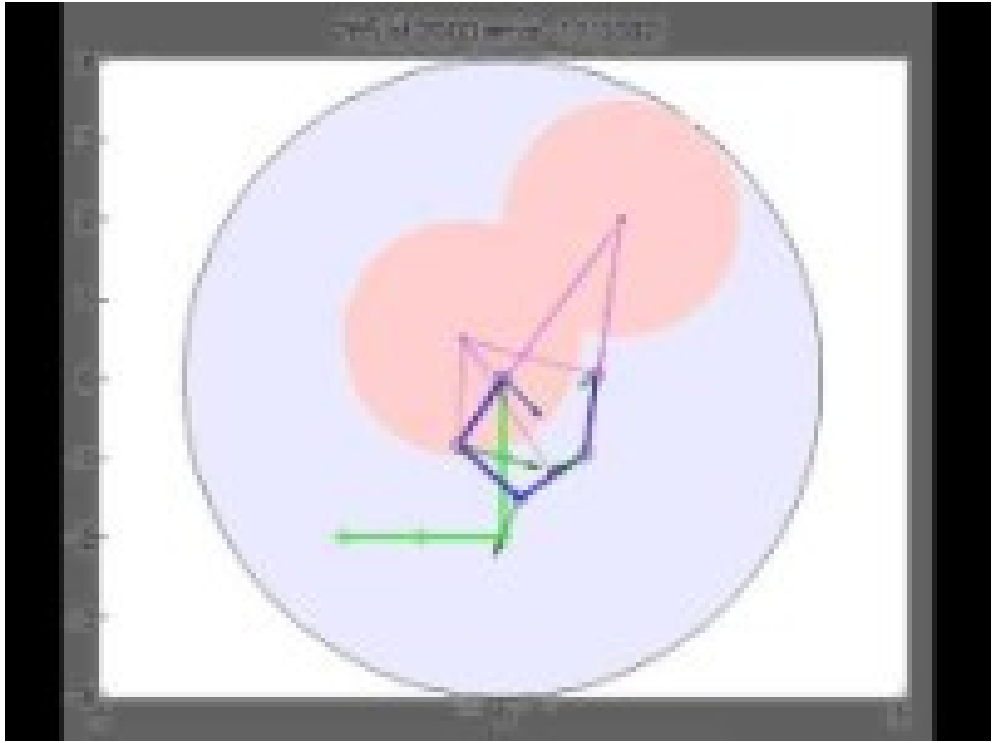
Adding Obstacles

- Pick several points on the manipulator
- Compute attractive and repulsive virtual forces for each
- Transform these forces into the configuration space and add them
- Use the resulting torques to move the robot (in its configuration space)



Be careful to use the correct Jacobian!
Thus far we have been using the Jacobian that concern the coordinate frame of the end-effector

MATLAB Demo



Aaron T. Becker's Robot Swarm Lab (2023). Robot Arm Potential Field Navigation (https://www.mathworks.com/matlabcentral/fileexchange/73617-robot-arm-potential-field-navigation), MATLAB Central File Exchange

JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

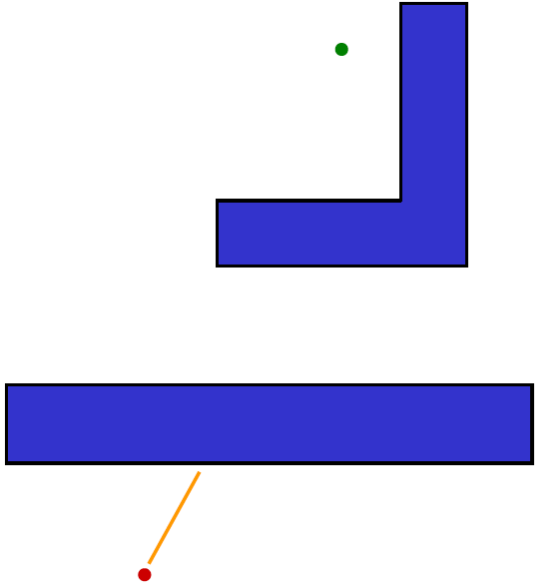
© The Johns Hopkins University 2024, All Rights Reserved.

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Robotics

Bug Algorithms

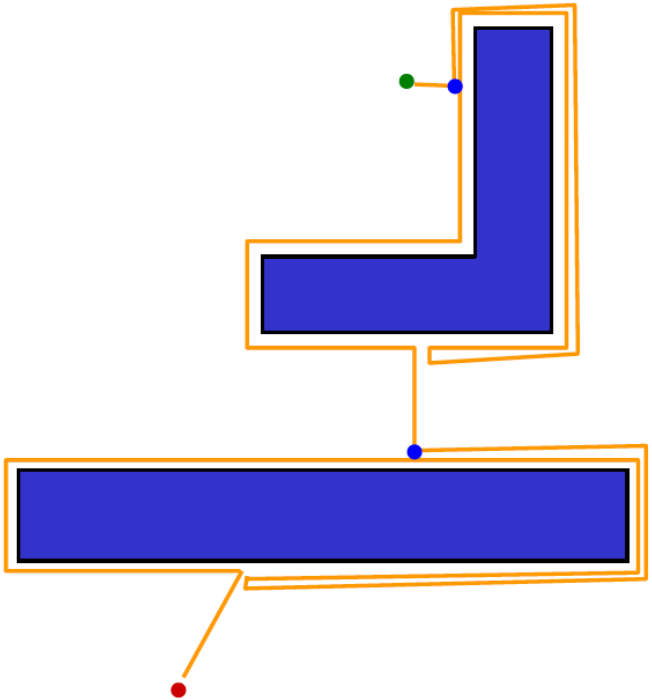
Bug 1



known direction to goal
otherwise local sensing
walls/obstacles & **encoders**

1) head toward goal
2) if an obstacle is encountered, circumnavigate it *and* remember how close you get to the goal.
3) return to that closest point (by wall-following) and continue

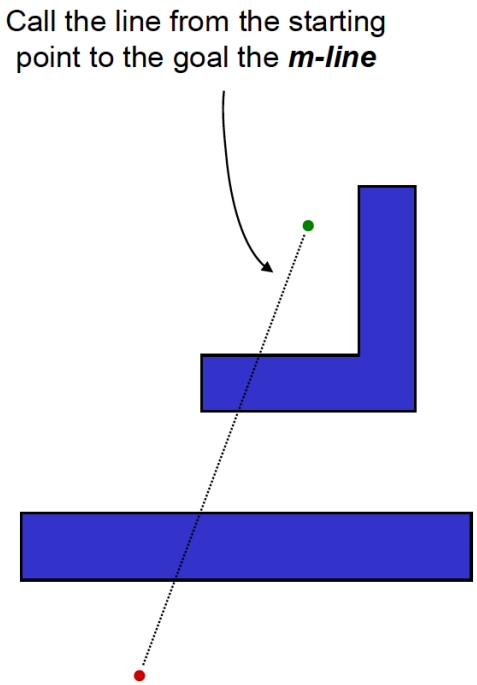
Bug 1



known direction to goal
otherwise local sensing
walls/obstacles & **encoders**

1) head toward goal
2) if an obstacle is encountered, circumnavigate it *and* remember how close you get to the goal.
3) return to that closest point (by wall-following) and continue

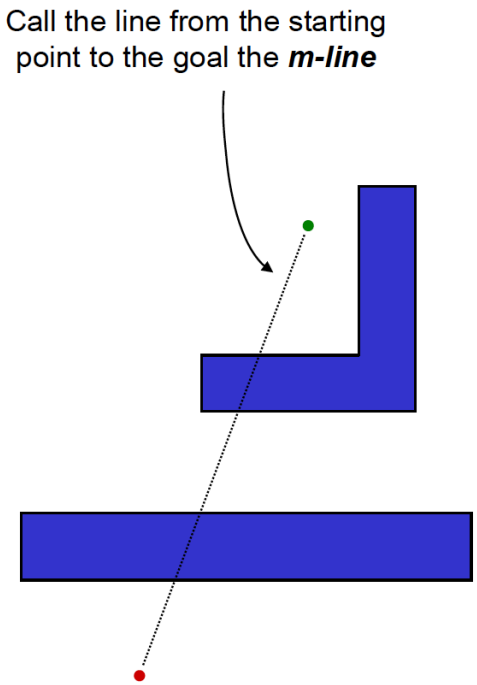
Bug 2



known direction to goal
otherwise local sensing
walls/obstacles & **encoders**

1) head toward goal
2) if an obstacle is in the way, follow it until you encounter the m-line again
3) Leave the obstacle and continue toward the goal

Bug 2

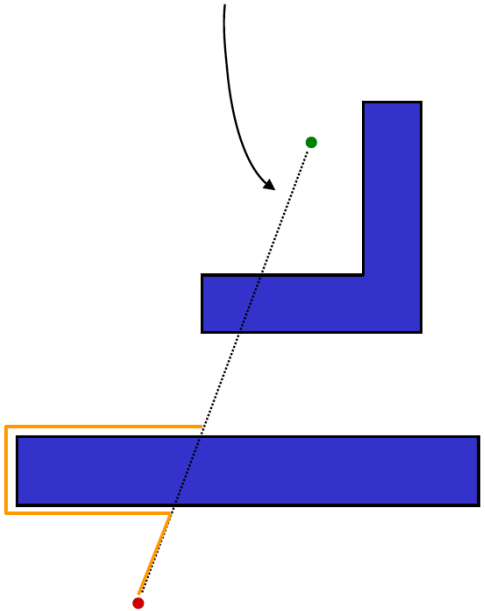


known direction to goal
otherwise local sensing
walls/obstacles & **encoders**

1) head toward goal
2) if an obstacle is in the way, follow it until you encounter the m-line again
3) Leave the obstacle and continue toward the goal

Bug 2

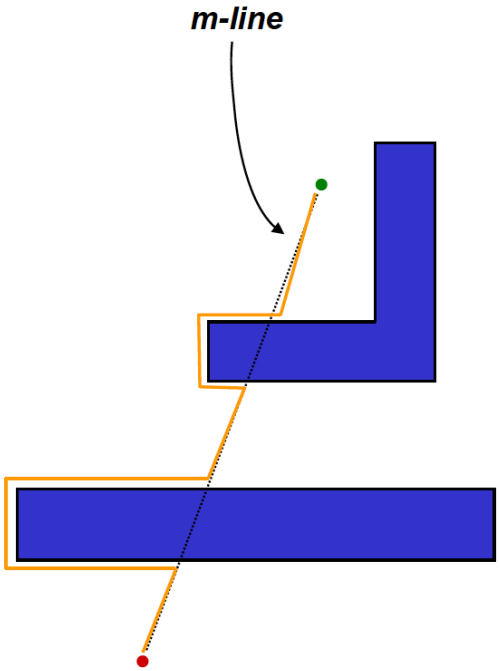
Call the line from the starting point to the goal the ***m-line***



known direction to goal
otherwise local sensing
walls/obstacles & **encoders**

1) head toward goal
2) if an obstacle is in the way, follow it until you encounter the m-line again
3) Leave the obstacle and continue toward the goal

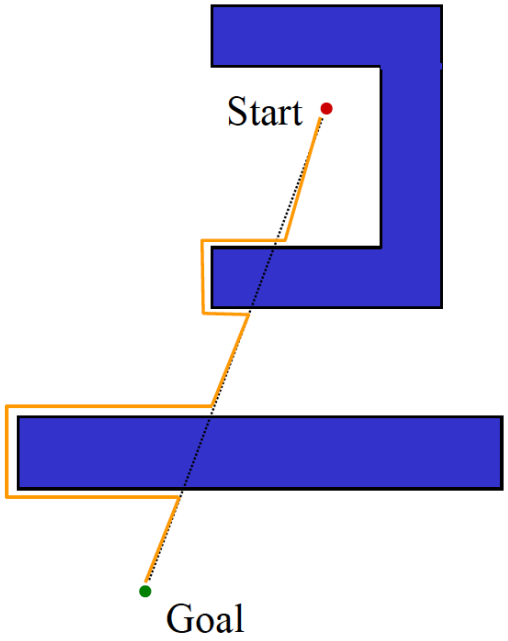
Bug 2 (4)



known direction to goal
otherwise local sensing
walls/obstacles & **encoders**

1) head toward goal
2) if an obstacle is in the way, follow it until you encounter the m-line again
3) Leave the obstacle and continue toward the goal

Bug 2 (5)

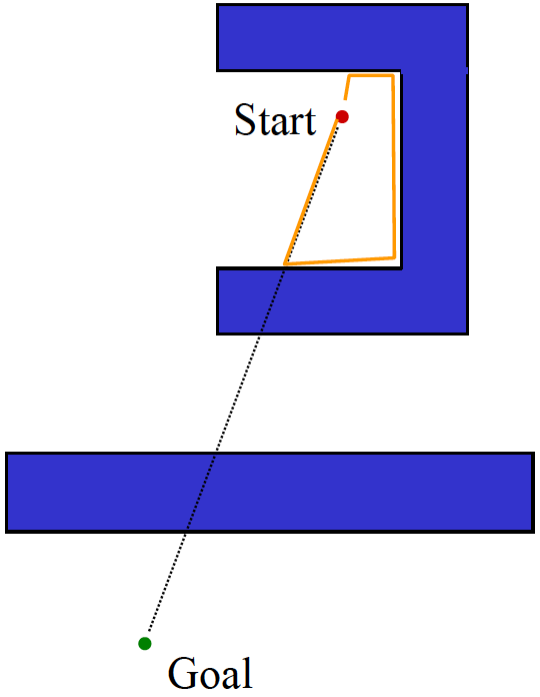


known direction to goal
otherwise local sensing
walls/obstacles & **encoders**

1) head toward goal
2) if an obstacle is in the way, follow it until you encounter the m-line again
3) Leave the obstacle and continue toward the goal

Better or worse than Bug 1?

Bug 2 (6)

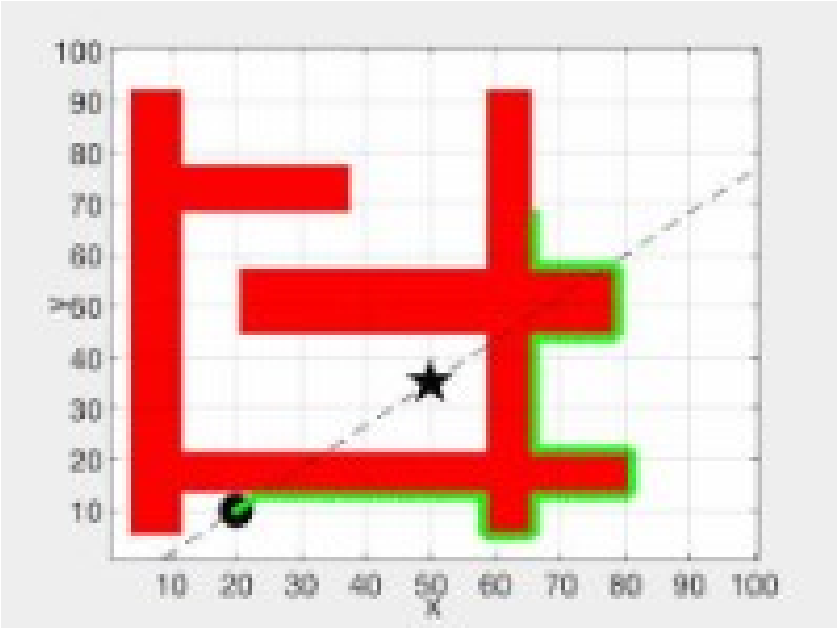


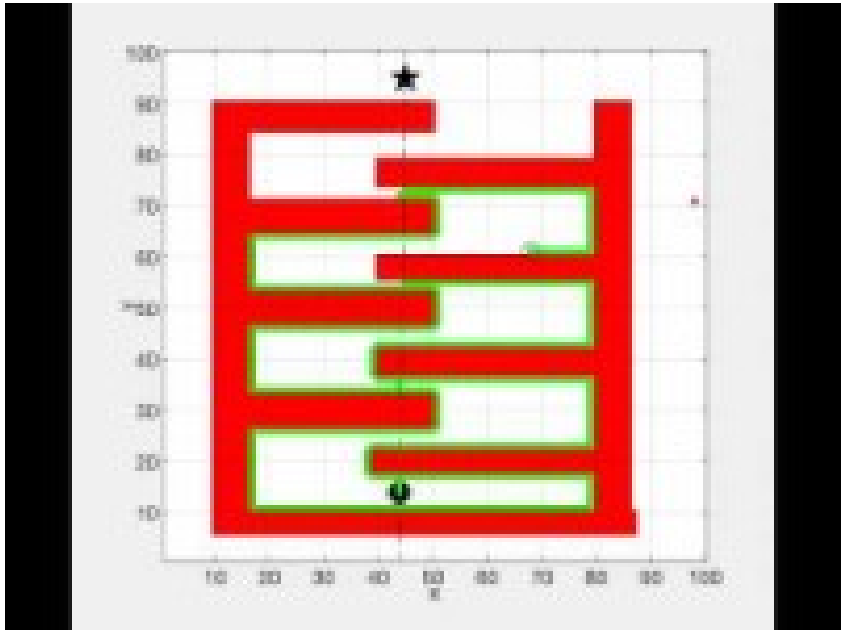
known direction to goal
otherwise local sensing
walls/obstacles & **encoders**

1) head toward goal
2) if an obstacle is in the way, follow it until you encounter the m-line again
3) Leave the obstacle and continue toward the goal

Better or worse than Bug 1?

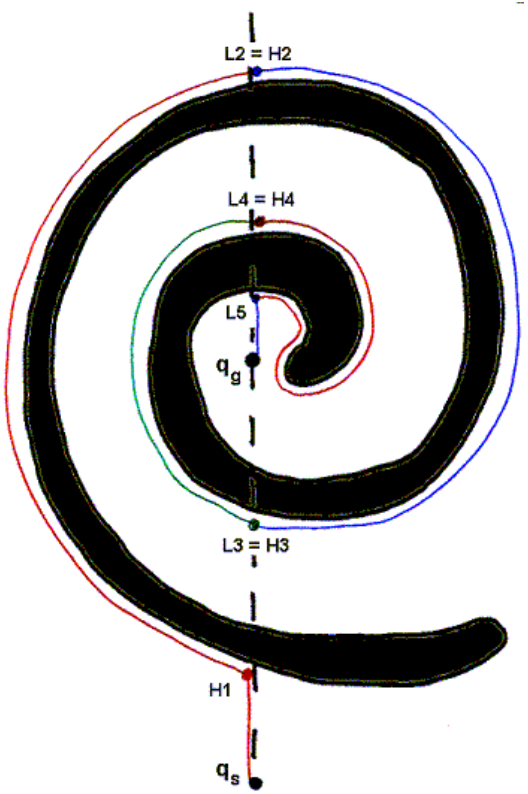
Bug Algorithm Demo

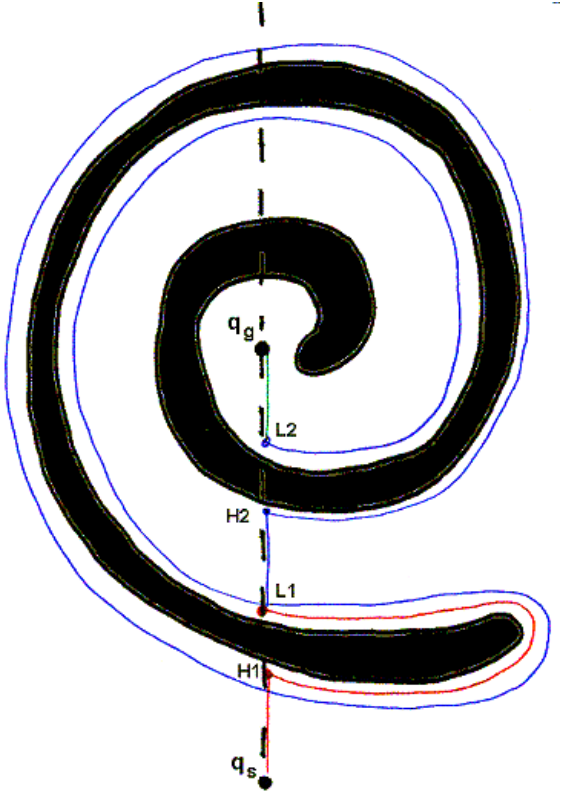




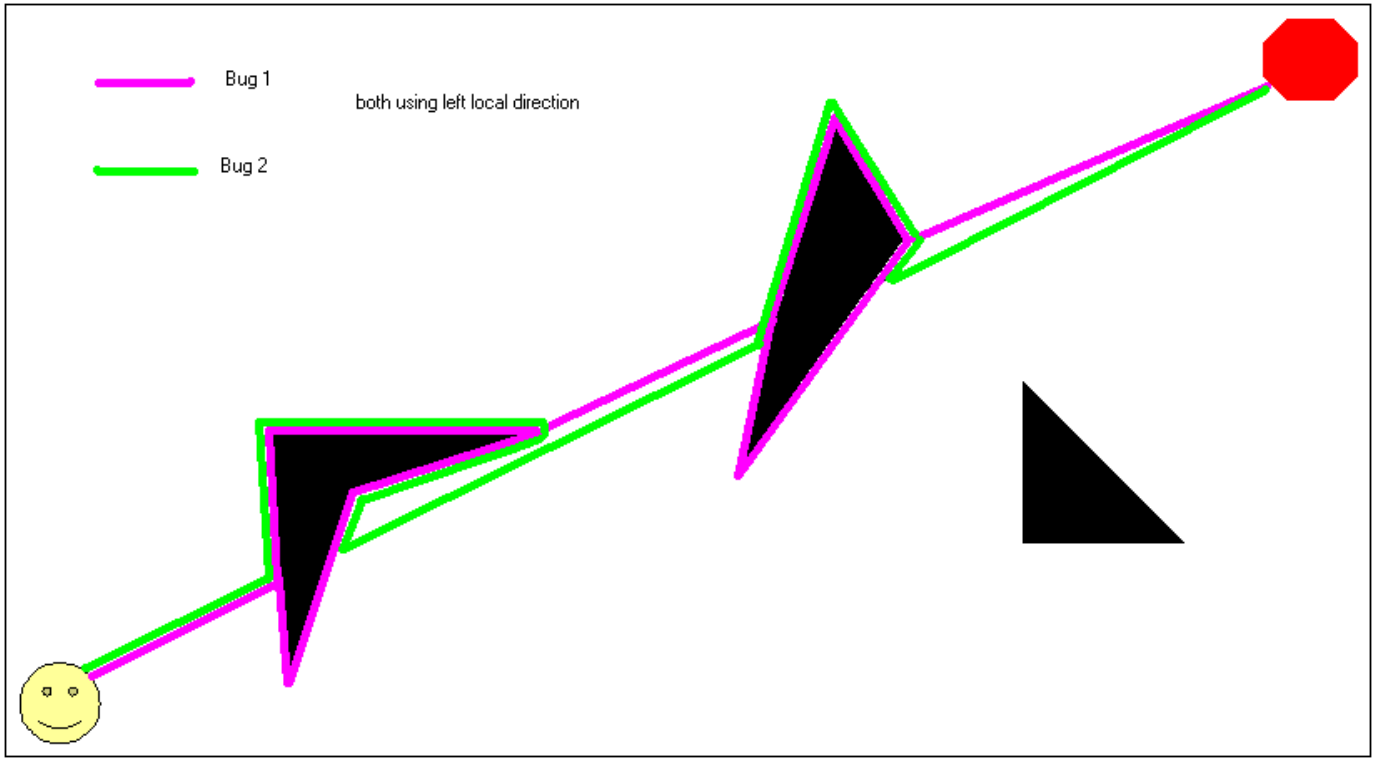
P.I. Corke, "Robotics, Vision & Control", Springer 2017, ISBN 978-3-319-54413-7

Bug 2 Spiral





Lumelsky Bug Algorithms



Lumelsky Bug Algorithms (cont.)

Unknown obstacles, known start and goal.

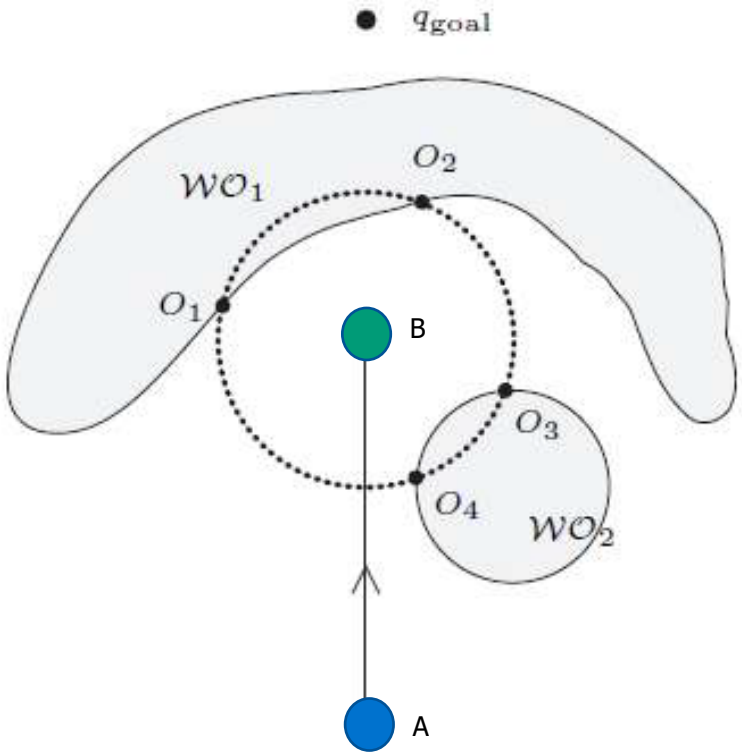
- Simple "bump" sensors, encoders.
- Choose arbitrary direction to turn (left/right) to make all turns, called "local direction"
- Motion is like an ant walking around:
  - In Bug 1 the robot goes all the way around each obstacle encountered, recording the point nearest the goal, then goes around again to leave the obstacle from that point
  - In Bug 2 the robot goes around each obstacle encountered until it can continue on its previous path toward the goal

TangentBug Algorithm

- What if instead of a touch sensor we had a limited range visual sensor?
- Search for regions of continuity
  - stretches of obstacles we can see broken up by gaps
- How much more efficiently can we reach the goal?

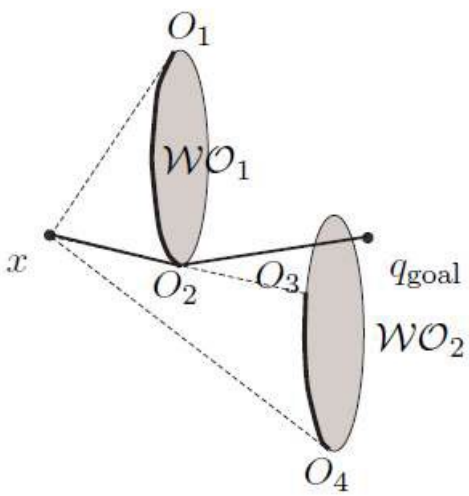
Motion-to-Goal Transition

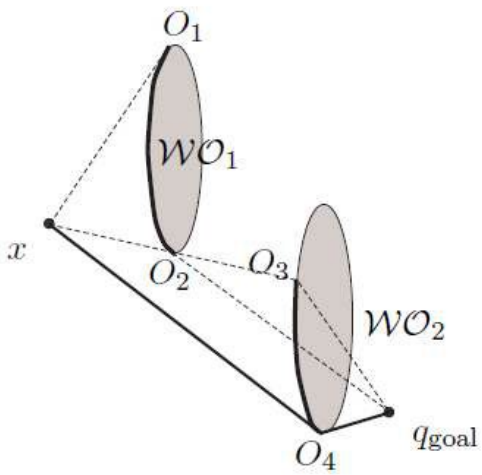
- The algorithm consists of two phases
  - Motion-to-Goal
  - Boundary Following
- In position A the robot cannot see the obstacle and assumes that a free path exists between itself and the goal
- In position B it now sees that the obstacle blocks its path



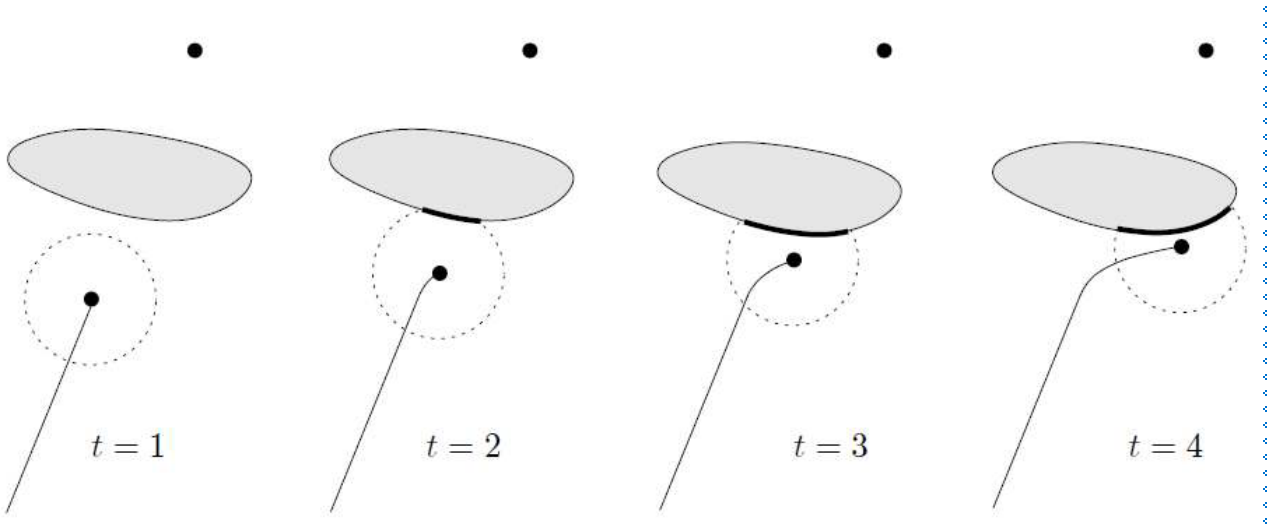
Minimum Distance Heuristic

- The robot will plan a path the goal assuming any space it cannot see is free (unoccupied)

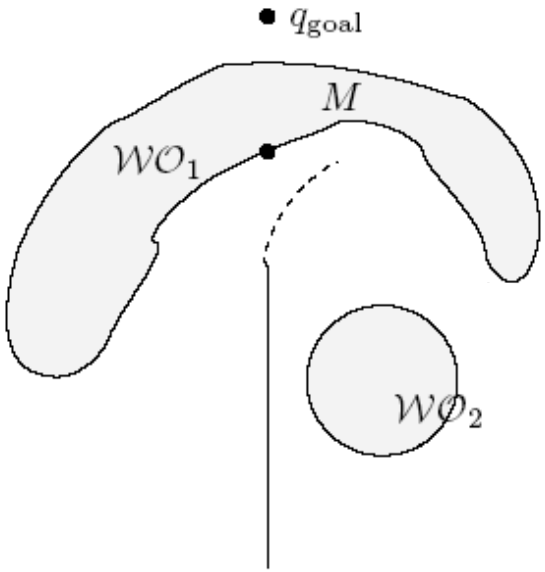




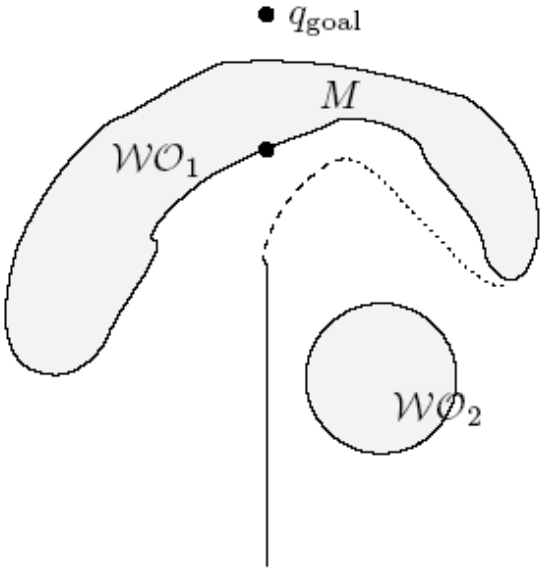
Motion to Goal Example



Behavior Transition Example

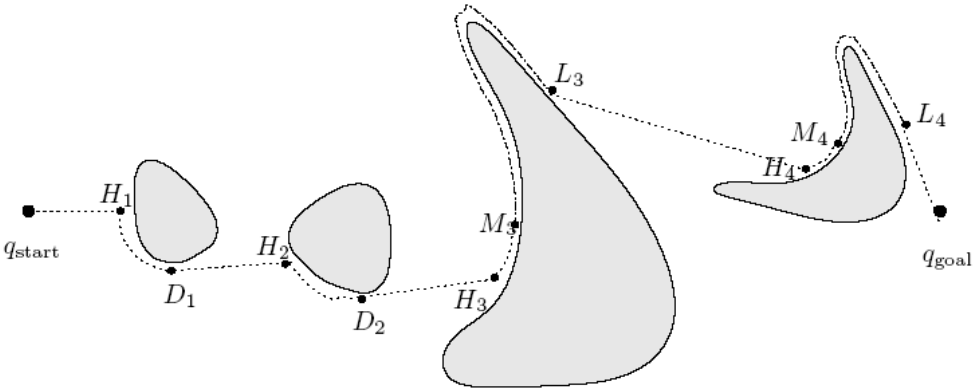


Motion-to-Goal

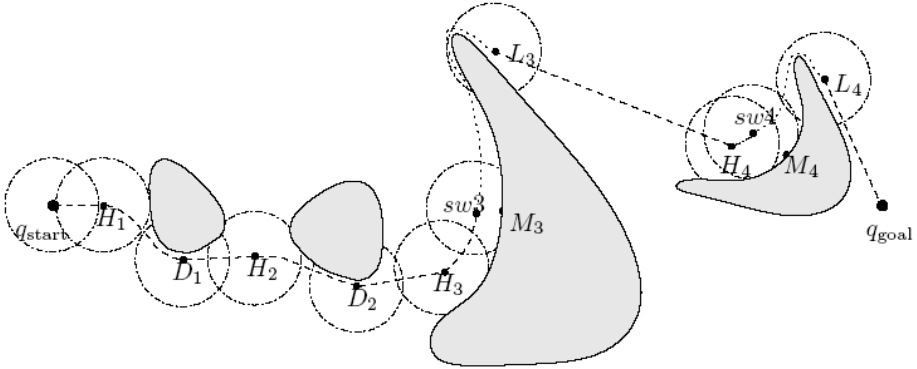


Boundary Following

Examples



Zero Sensor range



Limited Sensor Range

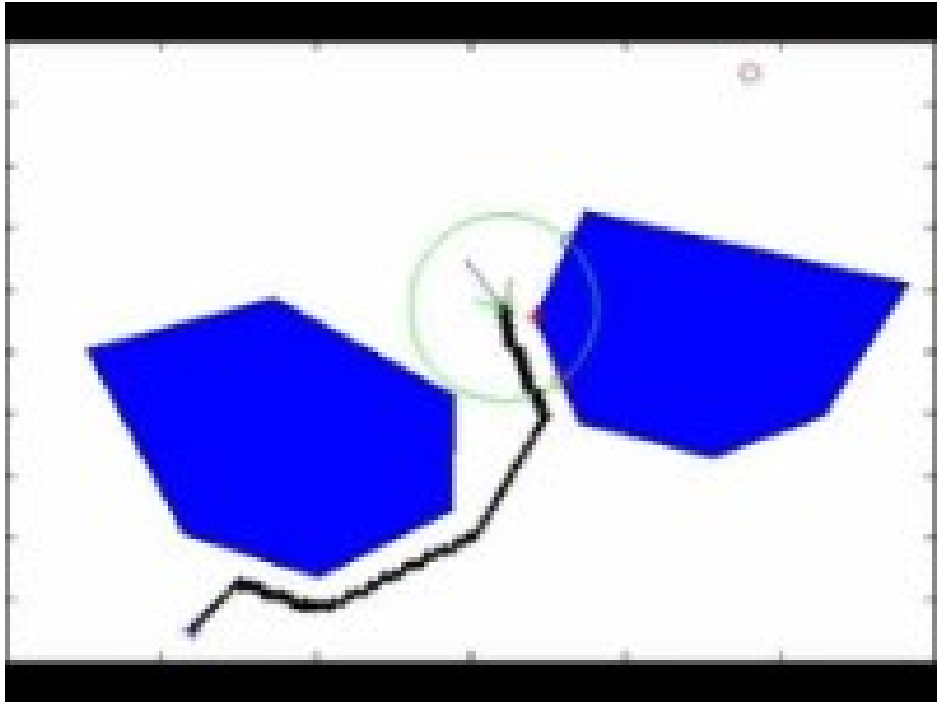
TangentBug Algorithm (cont.)

1) Repeat
   a) A) Compute continuous range segments in view
   b) Move toward n \in {T,O} that minimizes h(x,n) = d(x,n) + d(n,q_goal) until
      - Goal is encountered or
      - The value of h(x,n) begins to increase
2) Follow boundary continuing in same direction as before repeating
   a) Update {O}, d_leave and d_min until
      - Goal is reached
      - A complete cycle is performed (gual is unreachable)
      - d_leave < d_min

d_min is the shorted distance observed thus far between the sensed boundary of the obstacle and the goal

d_leave is the shortest distance between any point in the currently sensed environment and the goal

TangentBug Demo



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2024, All Rights Reserved.

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Robotics

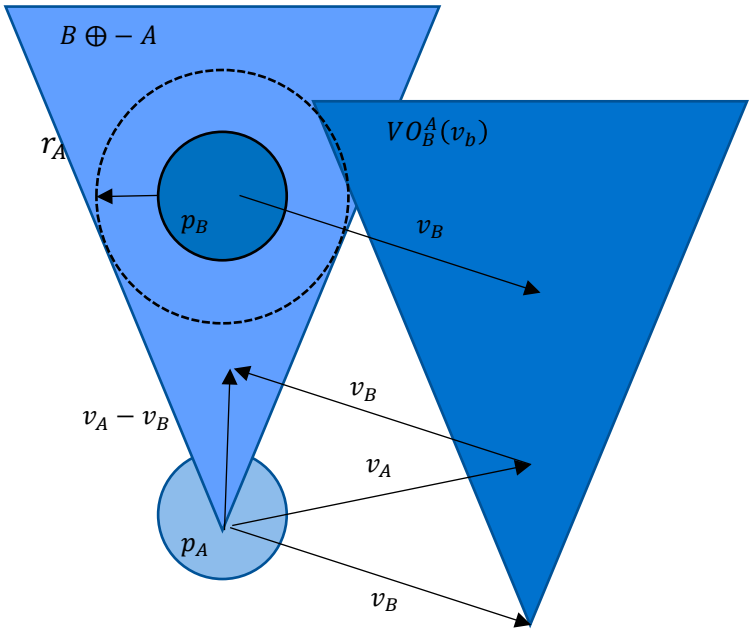
Velocity Obstacles

Motivation

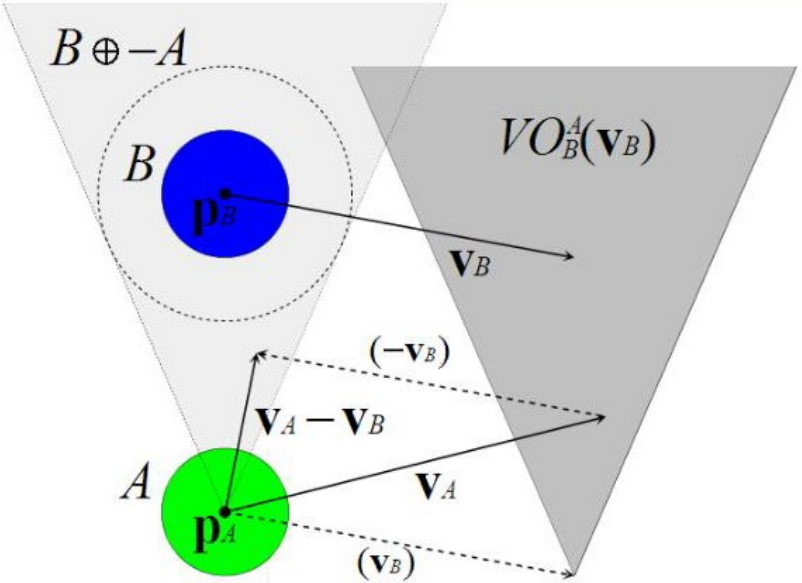
- Potential Fields are designed to handle static obstacles
  - Point or Particle Model
- How do we address moving obstacles? (assuming they maintain their current velocity)
  - Velocity Obstacles [Fiorinin, Shiller, 98]

Adapted from Van den Berg, J., Lin, M., & Manocha, D. (2008, May). Reciprocal velocity obstacles for real-time multi-agent navigation. In *2008 IEEE International Conference on Robotics and Automation* (pp. 1928-1935). IEEE.

Velocity Obstacles



Velocity Obstacles (cont.)



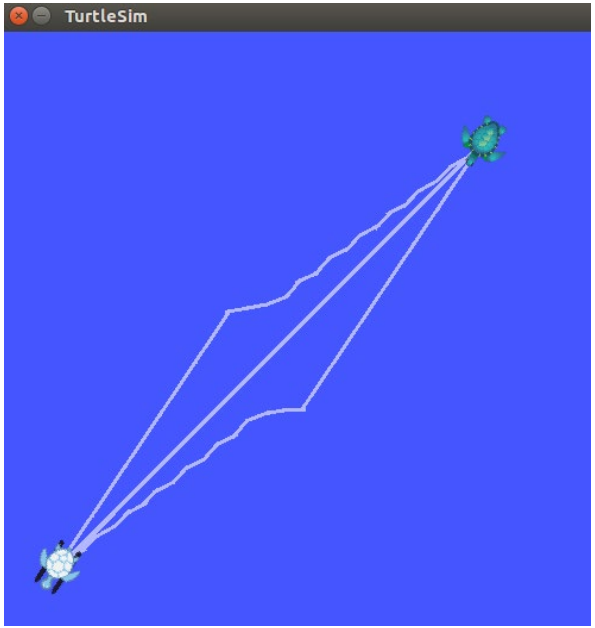
$$\gamma(p,v) = \{p + tv \mid t > 0\}$$

$$VO_B^A(v_b) = \{v_a \mid p_A, v_A - v_B\} \cap B \oplus -A \neq \varnothing$$

Adapted from Van den Berg et. Al (2008)

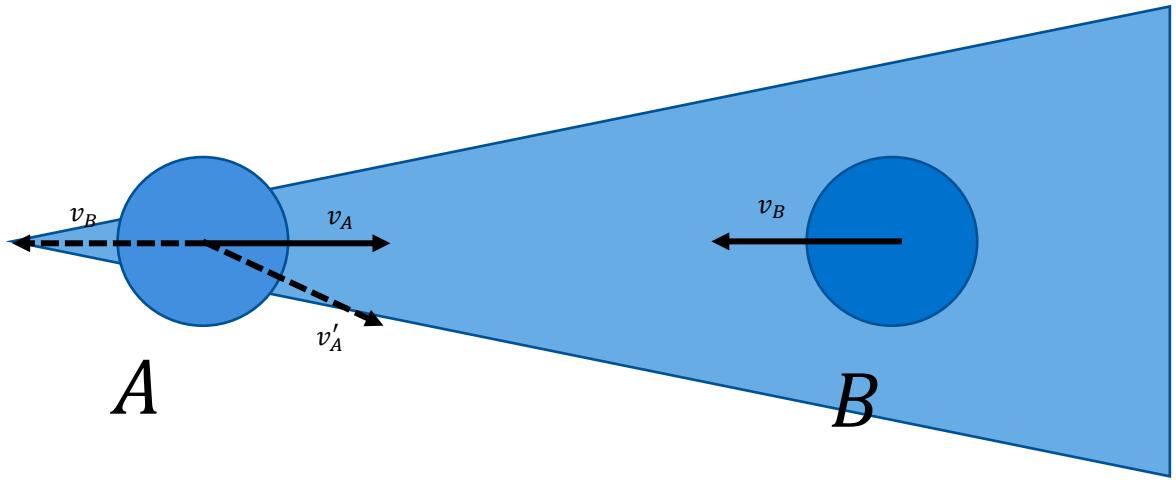
Multi-agent Setting

- In each cycle, select velocity outside velocity obstacle for any moving obstacle
- What happens if multiple agents are attempting to solve the problem at the same time?
  - Oscillations.

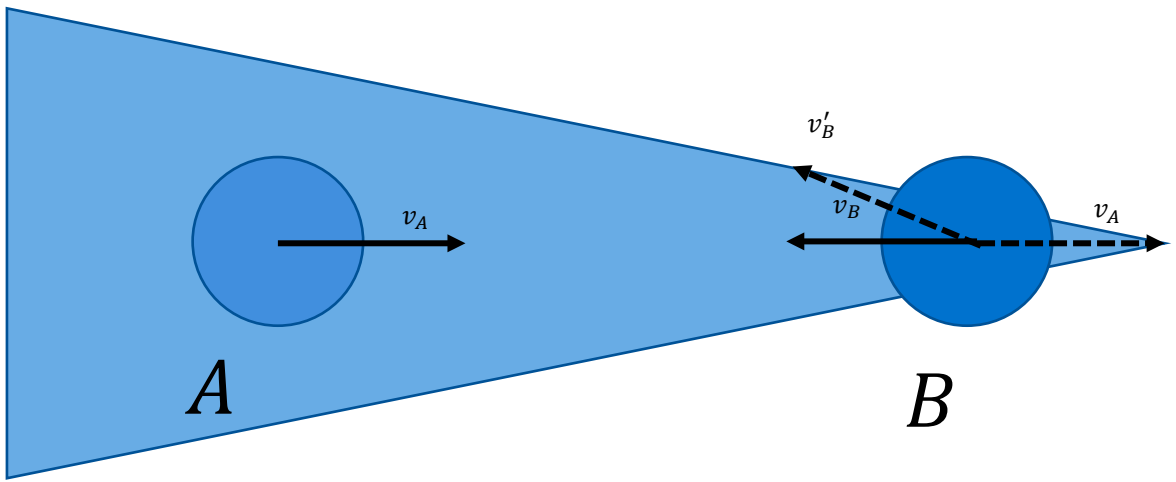


Oscillations (1)

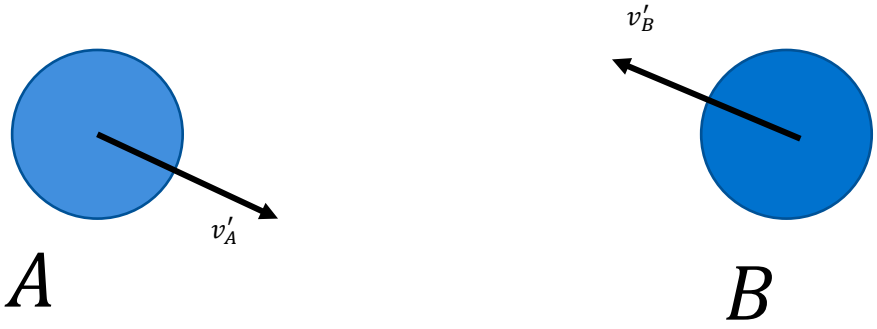
Oscillations (2)



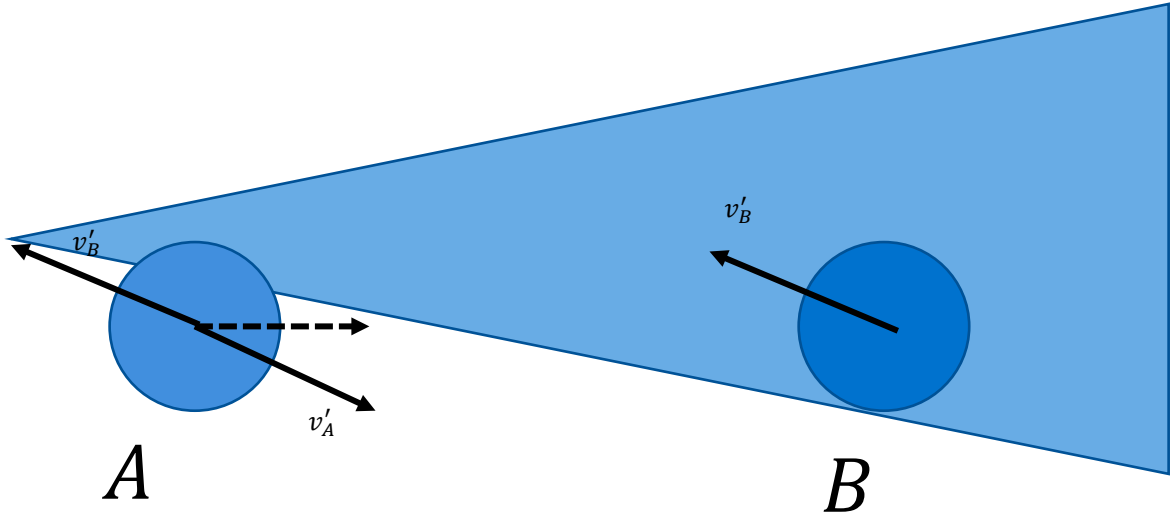
Oscillations (3)



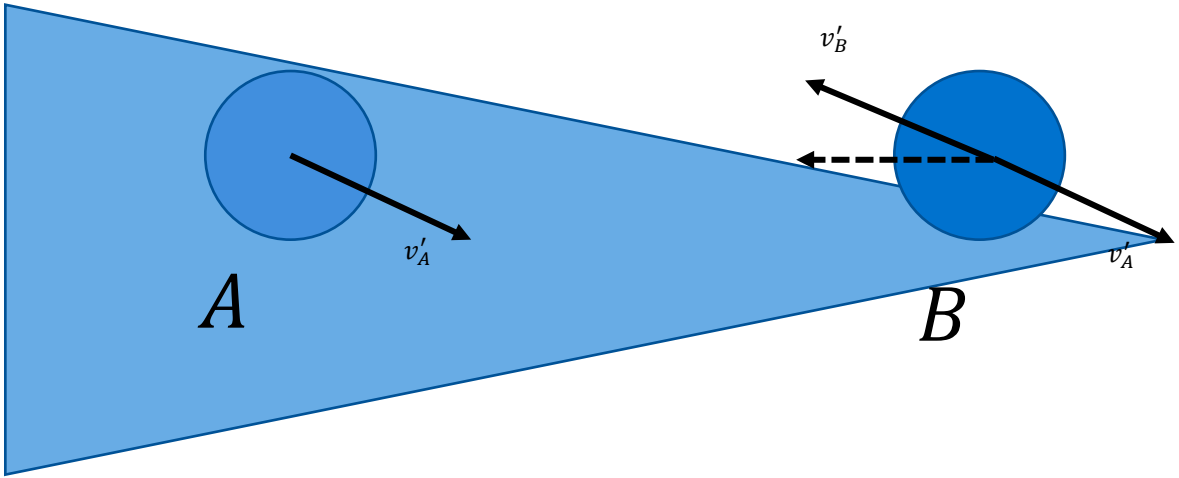
Oscillations (4)



Oscillations (5)

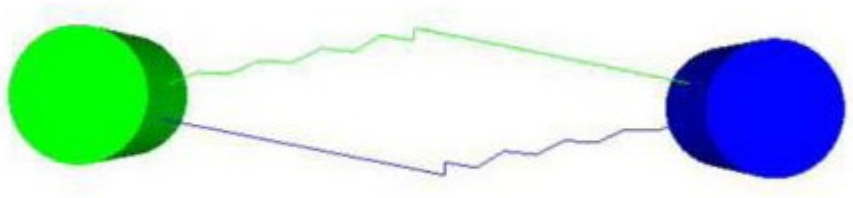


Oscillations (6)



Oscillations (7)

Oscillations (8)

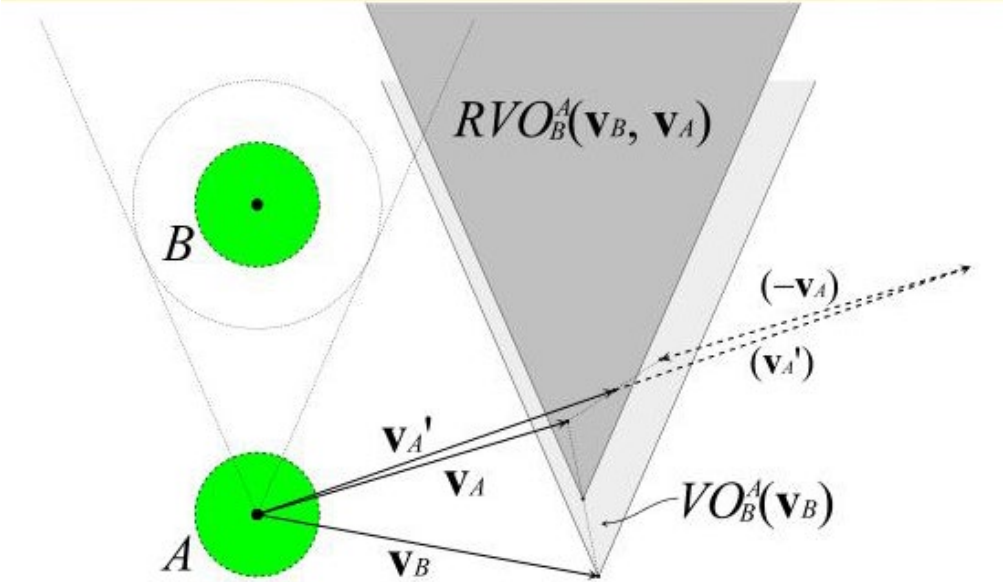


Adapted from Van den Berg et. Al (2008)

Reciprocal Velocity Obstacles

- Prevent Oscillations
- No communication among agents or global communication
- Simple idea: instead of choosing a new velocity outside the velocity obstacle, take the average of a velocity outside the velocity obstacle and the current velocity
- Formalized into Reciprocal Velocity Obstacles [Van den Berg, et.al. 2008]

Reciprocal Velocity Obstacle



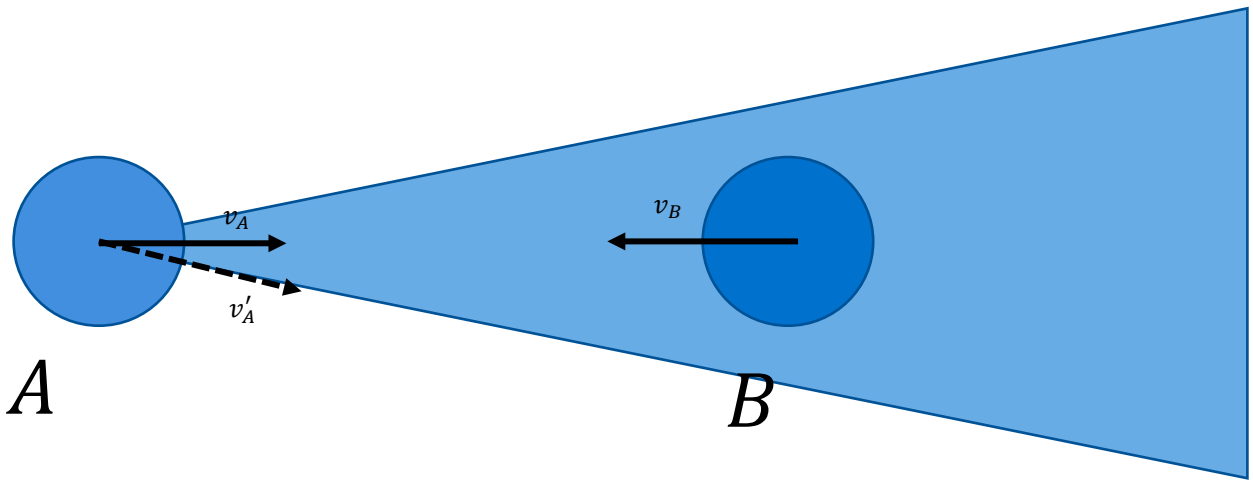
$$RVO_B^A(v_B, v_A) = \{v_A' \mid 2v_A' - v_A \in VO_B^A(v_B)\}$$

Adapted from Van den Berg et. Al (2008)

Oscillations (9)

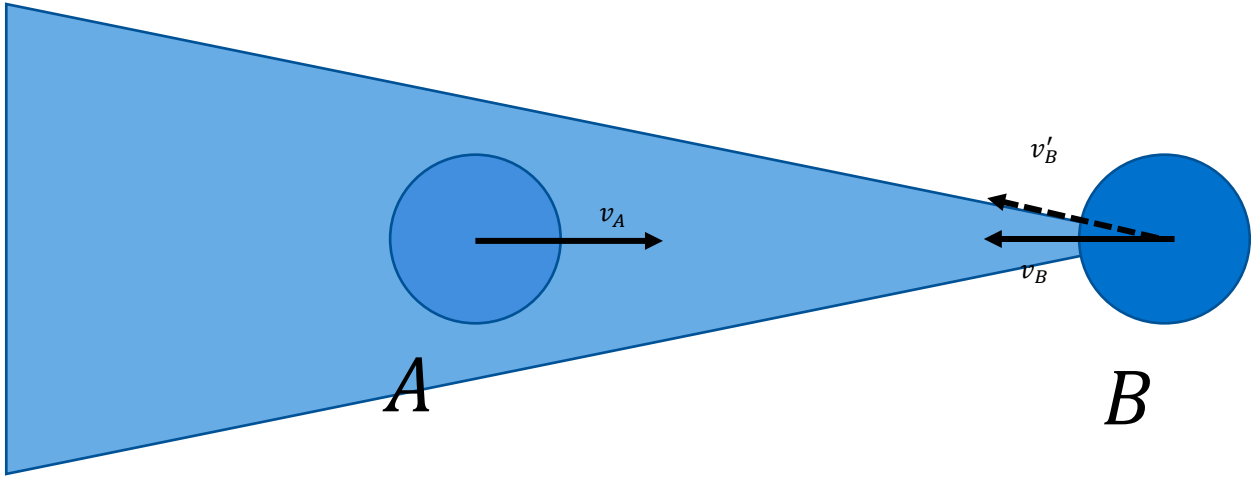
Adapted from Van den Berg et. Al (2008)

Oscillations (10)



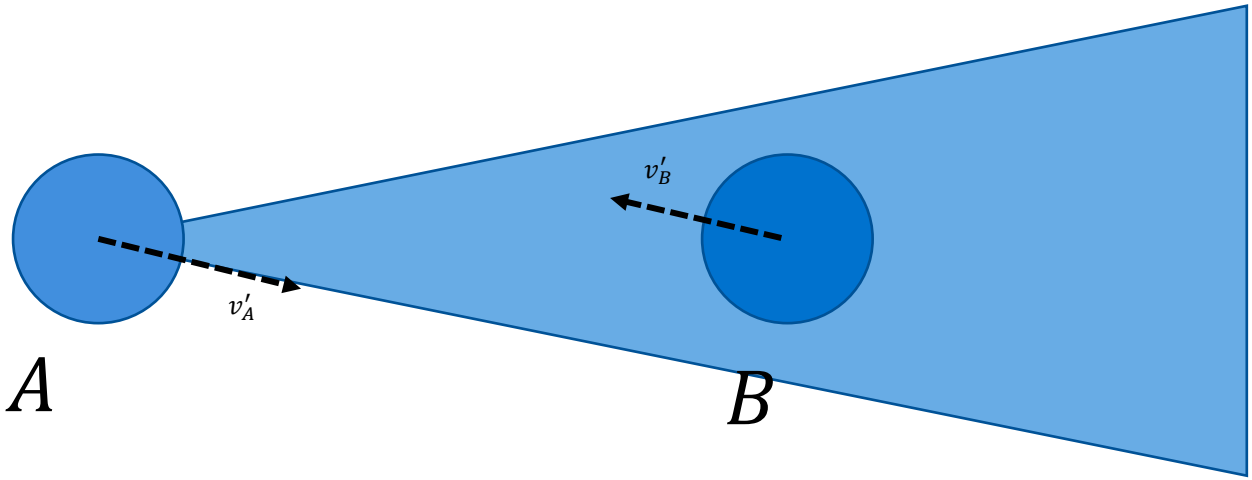
Adapted from Van den Berg et. Al (2008)

Oscillations (11)



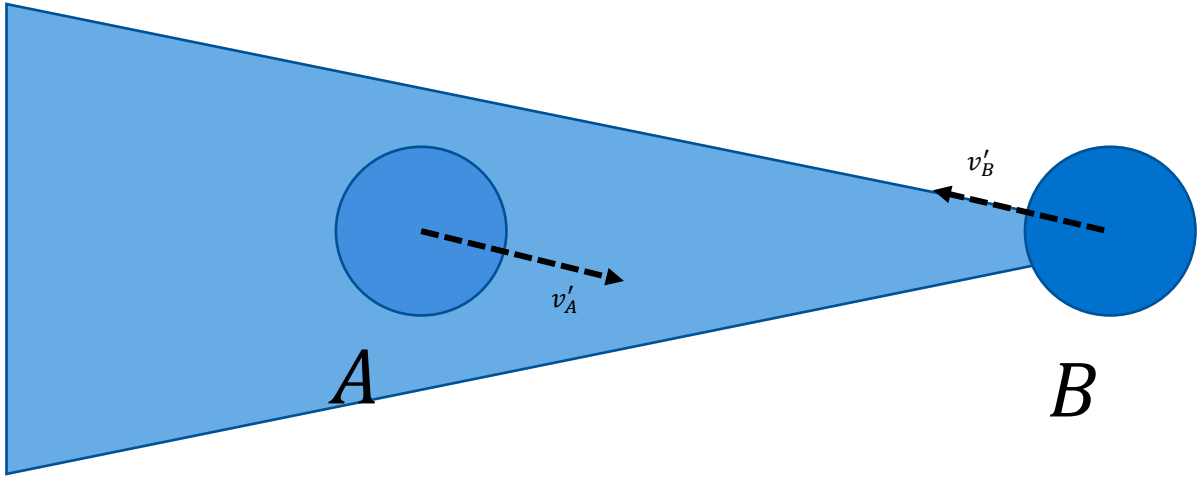
Adapted from Van den Berg et. Al (2008)

Oscillations (12)



Adapted from Van den Berg et. Al (2008)

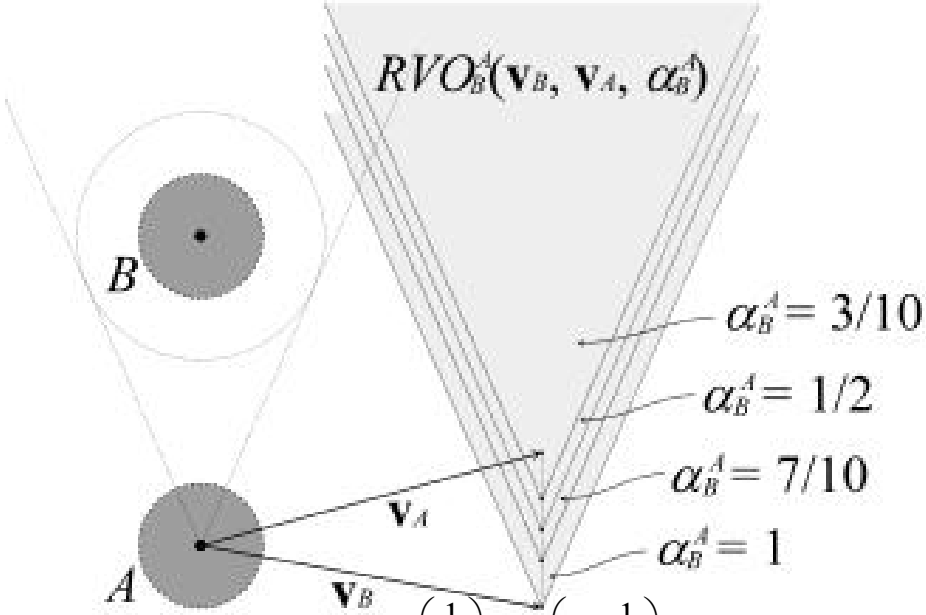
Oscillations (13)



Adapted from Van den Berg et. Al (2008)

Generalized RVOs

- Different distribution of effort in avoiding each other other than 50%
- Parameter α where
  - $\alpha \to 0$: no effort
  - $\alpha \to 1$: entirely responsible

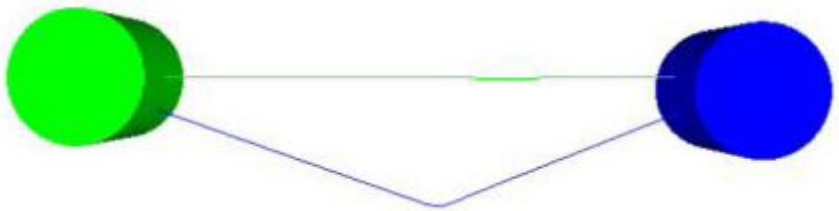


$$RVO_B^A(v_B, v_A, \alpha) = \left\{ v_A' \left| \left(\frac{1}{\alpha} \right) v_A' - \left(1 - \frac{1}{\alpha} \right) v_A \in VO_B^A(v_B) \right. \right.$$

$$RVO_B^A(v_B, v_A, \alpha) = \{ v_A' | \left(\frac{1}{\alpha} \right) v_A' - \left(1 - \frac{1}{\alpha} \right) v_A \in VO_B^A(v_B)$$

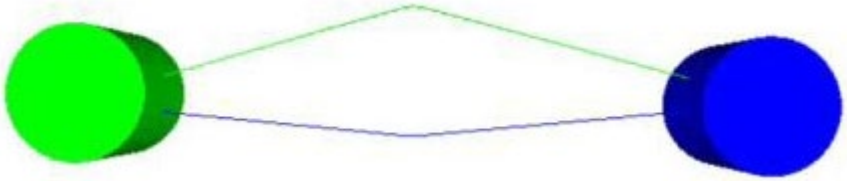
Adapted from Van den Berg et. Al (2008)

Example Values of α



$\alpha = 1$

$\alpha = 0.75$



$\alpha = 0.25$

$\alpha = 0$

Adapted from Van den Berg et. Al (2008)

Multi-Agent Navigation

- N agents $A_1, \ldots, A_N$ with positions $p_1, \ldots, p_N$ velocities $v_1, \ldots, v_N$ preferred speeds $v_1^{\text{pref}}, \ldots, v_N^{\text{pref}}$ and goals $g_1, \ldots, g_N$
- Time step Δt
- Each time step for each agent
  - Compute preferred velocity (global path planning)
  - Select new velocity
  - Update position of agent according to new velocity
- Outside the union of the reciprocal velocity obstacles, closest to the preferred velocity.
- Environment may become crowded: No valid velocity
- Solution: Select velocity inside RVO but penalize
  - Expected time of collision
  - Distance to preferred velocity
- Select velocity with minimum penalty

Adapted from Van den Berg et. Al (2008)

JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2024, All Rights Reserved.